

Lab2

Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins.

Theory

Maven is a powerful build automation and dependency management tool used primarily for Java projects, although it can be used for projects in other programming languages as well. It provides a comprehensive set of features to help manage the software development lifecycle and simplify the build process.

Maven Build Lifecycle

Maven has three built-in build lifecycles: clean, default, and site.

- **Clean Lifecycle:** Deletes any build output generated by previous builds.
- `clean`: Deletes any build output generated by previous builds.
- **Default Lifecycle:**
 - `validate`: Validates the project structure and verifies if all necessary information is available.
 - `compile`: Compiles the project's source code.
 - `test`: Runs unit tests against compiled source code.
 - `package`: Packages the compiled code into a distributable format (e.g., JAR, WAR).
 - `install`: Installs the package into the local repository for use as a dependency in other projects (.m2).
 - `deploy`: Deploys the package to a remote repository for sharing with other developers or environments (Push to Nexus).
- **Site Lifecycle:**
 - `site`: Generates project documentation and reports.
 - `site-deploy`: Deploys the generated documentation to a remote web server.

Maven – POM

- Project Object Model (POM) refers to the XML files with all the information regarding project and configuration details. It contains the project description, as well as details regarding the versioning and configuration management of the project.

- The XML file is in the project home directory. Maven searches for the POM in the current directory when any given task needs to be executed.
- Some of the configuration that can be specified in the POM are following – project dependencies, plugins, goals, build profiles, project version, developers, mailing list

Dependency management

Dependency management - this allows project authors to directly specify the versions of artifacts to be used when they are encountered in transitive dependencies or in dependencies where no version has been specified.

Maven Plugin

“Maven” is really just a core framework for a collection of Maven Plugins. In other words, plugins are where much of the real action is performed, plugins are used to: create jar files, create war files, compile code, unit test code, create project documentation, and on and on. Almost any action can think of performing on a project is implemented as a Maven plugin.

How to add Dependencies and Plugins in pom.xml

- The newly created project will have a pom.xml file in its root directory. This file is crucial for Maven configuration.

Adding Dependencies

- Open pom.xml.
- To add a dependency according to requirement , find the dependency information on Maven Repository (e.g., <https://mvnrepository.com/>).

Add the dependency snippet within the <dependencies> tag. For example:

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.2.3.RELEASE</version>
</dependency>
```

Adding Plugins

```
<plugins>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.8.1</version>
    <configuration>
```

```
<source>1.8</source>

<target>1.8</target>

</configuration>

</plugin>

</plugins>
```

Procedure to create Maven Project

➤ Create one folder in desktop with name “mavenProject_batch” AND copy this path

1. Creating a Maven Project using command prompt

1) open command prompt and change the directory as follows

```
Cd C:\Users\Desktop\mavenProject_batch
```

2) Verify JDK installation: Ensure you have a JDK installed (not just a JRE).

- Set JAVA_HOME: C:\Program Files\Java\jdk-xx.
- Verify Maven: Run “mvn -version” in your terminal to confirm it is pointing to the correct JDK path.

3) To create mvn project

```
mvn archetype:generate
```

4) Specify Project Details

- Enter the Group ID (usually your organization or package name)
- Enter the Artifact ID (the name of your project).
- Fill in the Version and Packaging (e.g., jar).
- Click Finish to create the project.

5) Open Java Code (App.java) File inside mavenProject_batch folder and paste the following

```
package com.yit.demo;

public class App {

    public int add(int a, int b) {

        return a + b;

    }

    public static void main(String[] args) {

        App app = new App();
```

```

        int result = app.add(2, 3);

        System.out.println("2 + 3 = " + result);

        System.out.println("Application executed successfully!");
    }
}

```

6) Open Java Code (AppTest.java) File inside mavenProject_batch folder and copy the following code

```

package com.yit.demo;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class AppTest {

    @Test
    public void testAdd() {
        App app = new App();
        int result = app.add(2, 3);
        System.out.println("Running test: 2 + 3 = " + result);
        assertEquals(5, result);
    }
}

```

7) To build and run this project, follow these steps:

1. Compile the Project

```
mvn compile
```

2. Run the Unit Tests

```
mvn test
```

3. **Package the project into a JAR**

```
mvn package
```

4. **Run the application (using JAR)**

```
java -cp target\exp2-1.0-SNAPSHOT.jar com.yit.demo.App
```


2. Creating a Maven Project using IDEs

Initial Set UP

- 1) Window->preference->java->installed JRE->add->jdk from directory->uncheck the JRE and select JDK
- 2) Window->preference->compiler->compiler compliance level->1.8
- 3) Window->preference->mavan->add the maven from program files->select maven

Creating Maven project

- 1) project->create maven project->select use default wokspace location->next->catalog->all catalog or maven-central->search for maven-archetype-simple->select org.apache.maven->next->group id and archetype->finish
- 2) You can manually navigate the **project folder** named call **exp2** and open the file pom.xml and copy the below code and paste it then save it.
 - **Open Java Code (App.java) File and paste it (code is similar to previous App.java)**
 - **Open Java Code (AppTest.java) File and copy the following code (code is similar to previous AppTest.java)**
- 3) Now click on the maven project name “exp2” and click on src/main/java then click on com.yit.exp2 package, now right click on App.java click on “Run as” Java application.
- 4) Next again click on the maven project name “exp2” and click on src/test/java then click on com.yit.exp2 package, now right click on AppTest.java click on “Run as” Junit Test.

Lab 3

Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation.

Create folder in desktop with name “gradle-project” inside that create folder with name “groovy”

Step 1: Create a new Project

Cd path of groovy folder

```
gradle init --type java-application
```

Step 2: build.gradle (Groovy DSL)

gradle-project -> groovy -> app -> build.gradle -> default code will be generated, compare and add the required line.

```
plugins {  
    id 'application'  
}  
  
application {  
    mainClass = 'com.yit.AdditionOperation'  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    testImplementation 'junit:junit:4.13.2'  
}  
  
test {  
    outputs.upToDateWhen { false }  
}
```

```

testLogging {
    events "passed", "failed", "skipped"
    exceptionFormat "full"
    showStandardStreams = true
}
}

```

Step 3: AdditionOperation.java(Change file name and update below code)

- After creating project **change the file name**.
- Inside gradle-project, manually navigate the folder path like **app/src/main/java/com/yit/**
- Change the file name **App.java** to **AdditionOperation.java**
- After then open that file and type the code for Addition of two numbers.

```

package com.example;

public class AdditionOperation {
    public static void main(String[] args) {
        double num1 = 5;
        double num2 = 10;
        double sum = num1 + num2;
        System.out.printf("The sum of %.2f and %.2f is %.2f\n", num1, num2, sum);
    }
}

```

Step 4: AdditionOperationTest.java (JUnit Test) (Change file name and update below code)

- After creating project **change the file name**.
- Inside gradle-project, manually navigate the folder path like **app/src/test/java/com/example/**
- Change the file name **AppTest.java** to **AdditionOperationTest.java**
- After then open that file and type the code for Addition of two numbers.

```

package com.example;

import org.junit.Test;

import static org.junit.Assert.*;

```

```

public class AdditionOperationTest {

    @Test

    public void testAddition() {

        double num1 = 5;

        double num2 = 10;

        double expectedSum = num1 + num2;

        double actualSum = num1 + num2;

        assertEquals(expectedSum, actualSum, 0.01);

    }

}

```

Step 5: Run Gradle Commands

- To **build** the project:

C:\Users\USER\Desktop\gradle-project> groovy>gradle build

- To **run** the project:

C:\Users\USER\Desktop\gradle-project>groovy>gradle run

- To **test** the project:

C:\Users\USER\Desktop\gradle-project>groovy>gradle test

Gradle Kotlin

In the desktop, inside gradle-project folder, create folder with name kotlin

Step 1: Create a new Project

Cd path of kotlin folder

Step 1: Create a new Project

gradle init --type java-application

Copy the src folder and build.gradle.kts outside app folder and delete app folder

Change the C:\Users\USER\Desktop\gradle-project\kotlin\src\main\java\com\yit

With C:\Users\USER\Desktop\gradle-project\kotlin\src\main\kotlin\com\yit

Step 3: Main.kt (Change file name and update below code)

- After creating project **change the file name**.
- Manually navigate the folder path like **src/main/kotlin/com/yit/**
- Change the file name **App.java** to **Main.kt**
- After then open that file and write the code for Addition of two numbers.

```
package com.yit
```

```
fun addNumbers(num1: Double, num2: Double): Double { // ← Top-level  
    return num1 + num2  
}
```

```
fun main() {  
    val num1 = 10.0  
    val num2 = 5.0  
    val result = addNumbers(num1, num2) // Can call from anywhere  
    println("The sum of $num1 and $num2 is: $result")  
}
```

Step 4: MainTest.kt (JUnit Test) (Change file name and update below code)

- After creating project **change the file name**.
- Manually navigate the folder path like **src/test/kotlin/com/yit/**
- Change the file name **MainTest.java** to **MainTest.kt**
- After then open that file and write the code for Addition of two numbers with Test block.

```
package com.yit
```

```
import org.junit.Test  
import org.junit.Assert.assertEquals
```

```
class MainTest {  
  
    @Test  
    fun testAddNumbers() {  
        val num1 = 10.0  
        val num2 = 5.0
```

```

    val result = addNumbers(num1, num2)

    assertEquals(
        "The sum of $num1 and $num2 should be 15.0",
        15.0,
        result,
        0.001
    )
}
}

```

➤ **Step5: make the changes to default generated build.gradle.kts file**

```

plugins {
    kotlin("jvm") version "1.9.24"
    application
}

group = "com.yit"
version = "1.0-SNAPSHOT"

repositories {
    mavenCentral()
}

dependencies {
    implementation(kotlin("stdlib"))
    testImplementation("junit:junit:4.13.2")
}

application {
    mainClass.set("com.yit.MainKt")
}

tasks.test {

```

```
useJUnit()

testLogging {
    events("passed", "failed", "skipped")
    exceptionFormat = org.gradle.api.tasks.testing.logging.TestExceptionFormat.FULL
    showStandardStreams = true
}

outputs.upToDateWhen { false }
}

java {
    toolchain {
        languageVersion.set(JavaLanguageVersion.of(17))
    }
}
```

Step 5: Run Gradle Commands

gradle build

- To **run** the project:

gradle run

- To **test** the project:

gradle test

Lab-4

Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle

Step 1: Creating a Maven Project

- Already we have created maven project in experiment 2. Go to experiment 2 pom.xml folder. In the folder pom.xml file, src folder and .mvn and target folder should be there.
- Copy the path of pom.xml. Open the command prompt. Change directory to pom.xml folder.

Step 2: Migrate the Maven Project to Gradle

1. **Initialize Gradle:** Navigate to the project directory (**gradle-example**) and run:
gradle init
 - It will ask **Found a Maven build. Generate a Gradle build from this? (default: yes) [yes, no]**
 - Type **Yes**
 - **Select build script DSL:**
 - 1: Kotlin
 - 2: Groovy
 - Enter selection (default: Kotlin) [1..2]
 - Type **2**
 - **Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no]**
 - Type **No**
2. Navigate the project folder and open **build.gradle** file then add the below code and save it.

```
plugins {  
  
    id 'java-library'  
  
    id 'maven-publish'  
  
}  
  
repositories {
```



```

mavenLocal()

maven {

    url = uri('https://repo.maven.apache.org/maven2/')

}

}

dependencies {

    testImplementation 'junit:junit:3.8.1'

}

group = 'com.yit'

version = '1.0-SNAPSHOT'

description = 'maven-demo'

java.sourceCompatibility = JavaVersion.VERSION_1_8

publishing {

    publications {

        maven(MavenPublication) {

            from(components.java)

        }

    }

}

task run(type: JavaExec) {

    main = 'com.yit.App'

    classpath = sourceSets.main.runtimeClasspath }

```

Step 3: Run the Gradle Project

- **Build the Project:** In the project directory (gradle-example), run the below command to build the project:

gradlew build

- **Run the Application:** Once the build is successful, run the application using below command:

gradlew run

Step 4: Verify the Migration

- **Compare the Output:** Make sure that both the **Maven** and **Gradle** builds produce the same output:

- **Gradle Output:**

- **Maven Output:**

mvn clean install

mvn exec:java -Dexec.mainClass="com.yit.demo.App"

Lab5

Introduction to Jenkins: What is Jenkins? Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use.

Introduction to Jenkins

What is Jenkins?

Jenkins is an open-source automation server widely used in the field of Continuous Integration (CI) and Continuous Delivery (CD). It allows developers to automate the building, testing, and deployment of software projects, making the development process more efficient and reliable. It provides a web-based interface and supports a wide range of plugins for integrating with different tools and technologies.

Key features of Jenkins:

- **CI/CD:** Jenkins supports Continuous Integration and Continuous Deployment, allowing developers to integrate code changes frequently and automate the deployment of applications.
- **Plugins:** Jenkins has a vast library of plugins that can extend its capabilities. These plugins integrate Jenkins with version control systems (like Git), build tools (like Maven or Gradle), testing frameworks, deployment tools, and much more.
- **Pipeline as Code:** Jenkins allows the creation of pipelines using Groovy-based DSL scripts or YAML files, enabling version-controlled and repeatable pipelines.
- **Cross-platform:** Jenkins can run on various platforms such as Windows, Linux, macOS, and others.

Installing Jenkins

Jenkins can be installed on local machines, on a cloud environment, or even in containers. Here's how you can install Jenkins in Window local System environments:

1. Installing Jenkins Locally

1. Prerequisites:

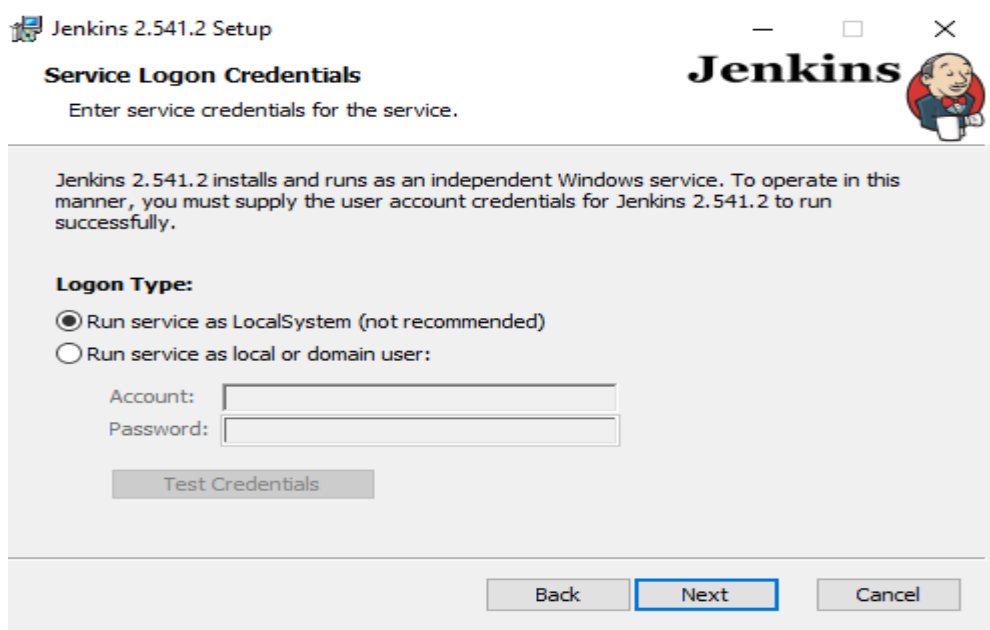
- Ensure that **Java (JDK)** is installed on your system. **Jenkins requires Java 21.** If not then [click here](#).
- You can check if Java is installed by running **java -version** in the terminal.

2. Install Jenkins on Window System):

- Download the Jenkins Windows installer from the [official Jenkins website](#).



- Run the installer and follow the on-screen instructions. While installing choose **login system: run service as LocalSystem (not recommended)**.



- After then use **default port** or you can **configure you own port like I'm using port 3030** then click on test and next.

a) Using Command Prompt (check any process is running on port 3030)

Open **Command Prompt** as administrator and run:

```
netstat -ano
```

```
C:\Users\Student>netstat -ano

Active Connections

Proto Local Address           Foreign Address         State       PID
TCP   0.0.0.0:135               0.0.0.0:0               LISTENING   8
TCP   0.0.0.0:445               0.0.0.0:0               LISTENING   4
TCP   0.0.0.0:623               0.0.0.0:0               LISTENING   3472
TCP   0.0.0.0:1521              0.0.0.0:0               LISTENING   3352
TCP   0.0.0.0:2343              0.0.0.0:0               LISTENING   4140
TCP   0.0.0.0:3306              0.0.0.0:0               LISTENING   4668
TCP   0.0.0.0:3580              0.0.0.0:0               LISTENING   3308
TCP   0.0.0.0:5040              0.0.0.0:0               LISTENING   7140
TCP   0.0.0.0:7680              0.0.0.0:0               LISTENING   6276
TCP   0.0.0.0:16992             0.0.0.0:0               LISTENING   3472
TCP   0.0.0.0:33060             0.0.0.0:0               LISTENING   4668
```

Explanation:

- `netstat` → Network statistics
- `-a` → Shows all connections and listening ports
- `-n` → Shows addresses and port numbers in numeric form
- `-o` → Shows the **Process ID (PID)** using each port

You'll see output like:

Proto	Local Address	Foreign Address	State	PID
TCP	0.0.0.0:80	0.0.0.0:0	LISTENING	1234
TCP	127.0.0.1:3306	0.0.0.0:0	LISTENING	5678

Find the process using a port:

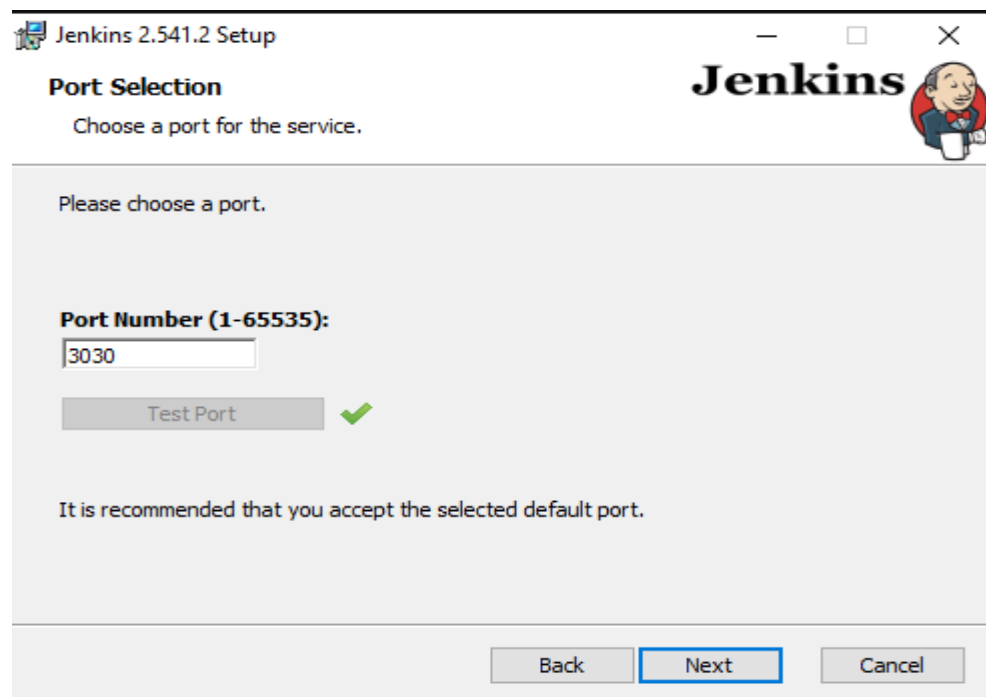
```
tasklist /FI "PID eq 1234"
```

```
C:\Users\Student>tasklist /FI "PID eq 3352"

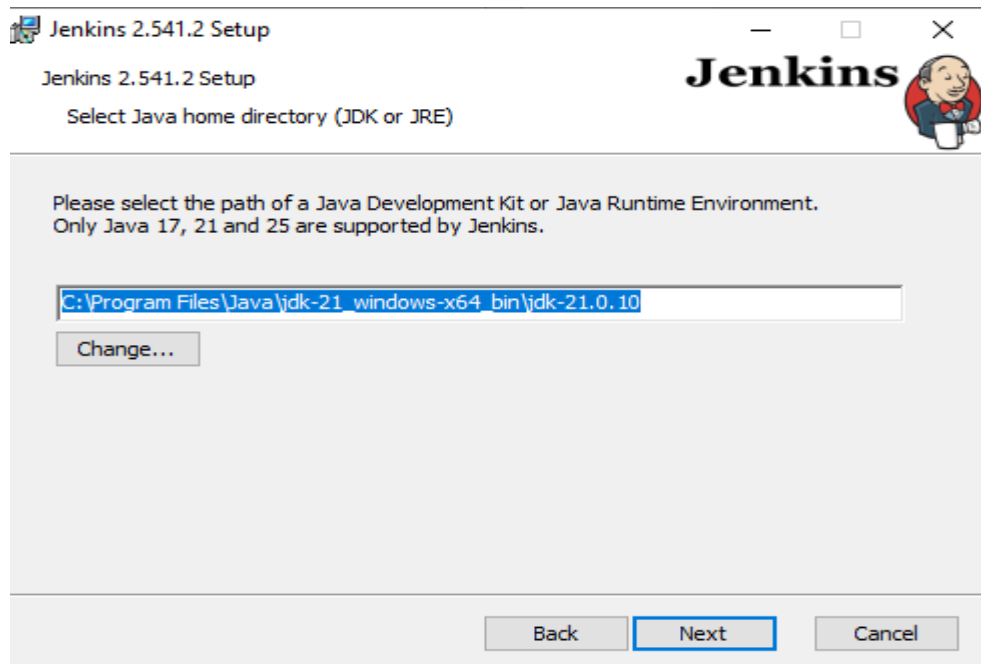
Image Name                        PID Session Name        Session#    Mem Usage
=====
TNSLNR.EXE                        3352 Services              0         10,748 K

C:\Users\Student>tasklist /FI "PID eq 8080"
INFO: No tasks are running which match the specified criteria.

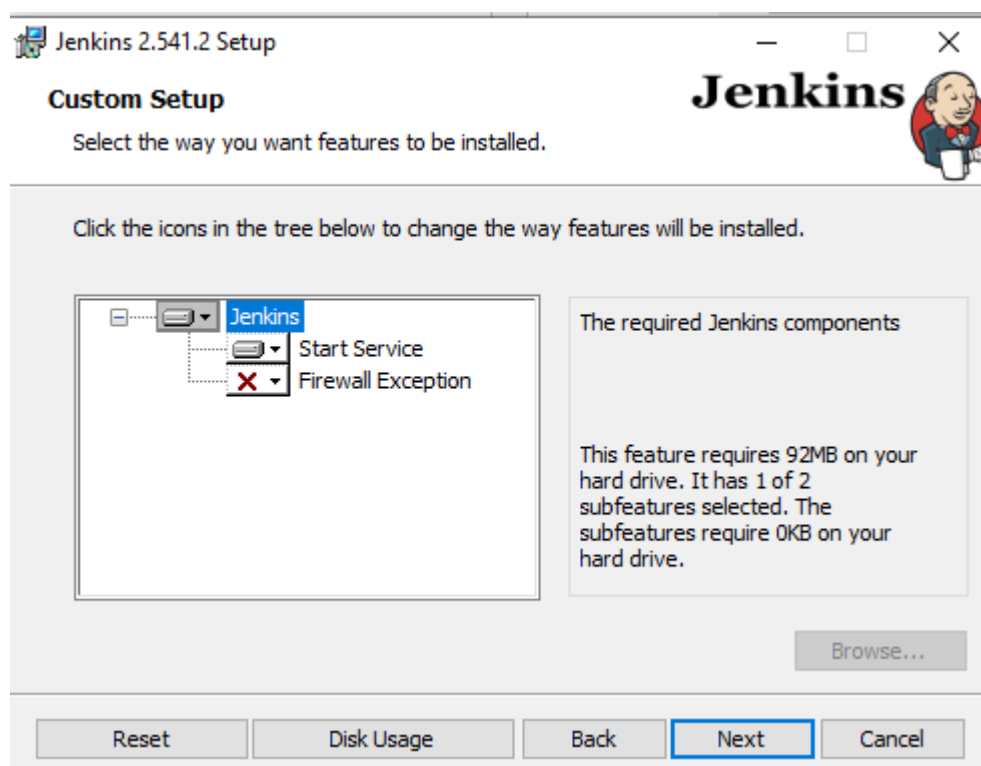
C:\Users\Student>tasklist /FI "PID eq 3030"
INFO: No tasks are running which match the specified criteria.
```

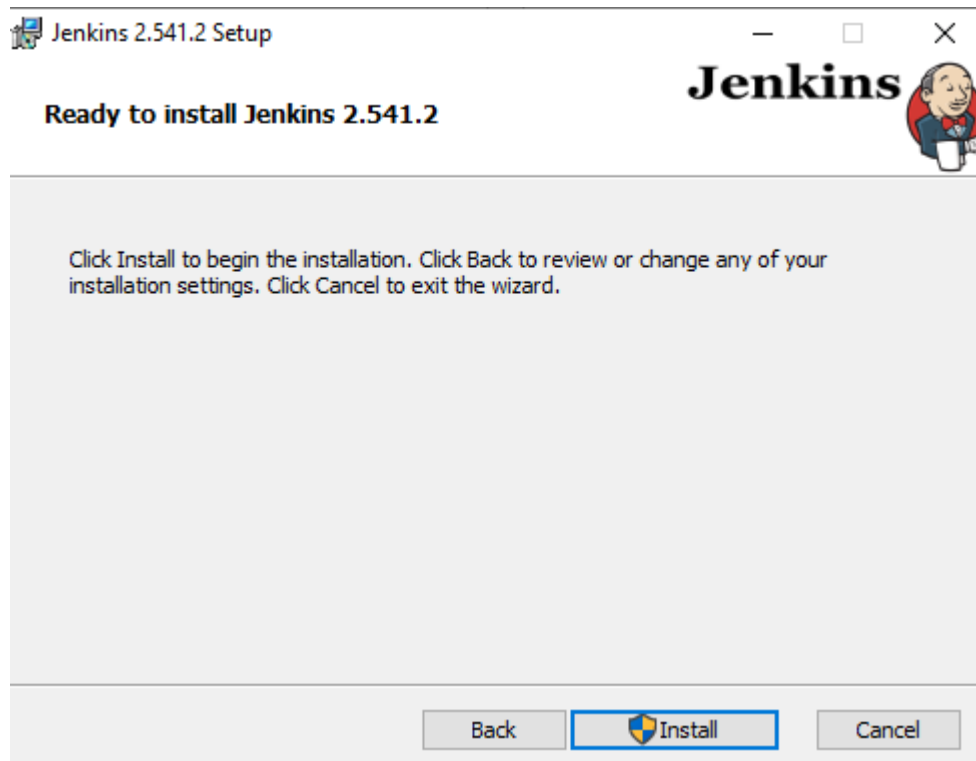


- After then **change the directory** and choose **java jdk-21** path look like **C:\Program Files\Java\jdk-21**.



- After then **click next, next** and then it will **ask permission** click on **yes** and it will start installing.

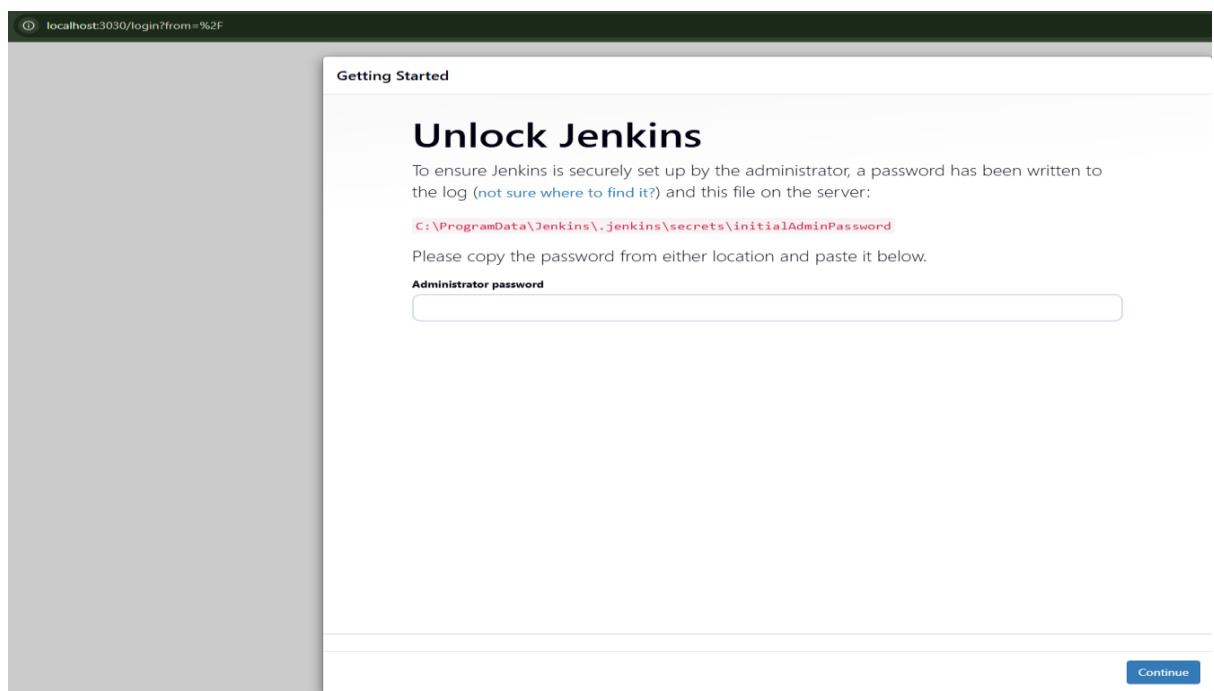




- After successfully installed, Jenkins will be **running on port either default port or chosen port 3030** by default (you can access it in your browser at **http://localhost:8080**) or **http://localhost:3030**.

2. Jenkins Setup in browser:

- After opening browser by visiting your local address the browser should look like below screenshot.



- It will ask **administrator password** so you have to **navigate the above highlighted path** and open that **initialAdminPassword** in **notepad** or any software to **see the password**.

```
C:\Users\Student>type C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
```

- Just **copy that password and paste it** and **click on continue**.
- It will ask to **customize Jenkins** so **click on install suggested plugin** it will **automatically install all required plugin**.

Getting Started

Getting Started



☒ Folders	☒ OWASP Markup Formatter	☐ Build Timeout	☐ Credentials Binding	** commons-lang3 v3.x Jenkins API ** Ionicons API Folders OWASP Markup Formatter ** ASM API
☐ Timestampers	☐ Workspace Cleanup	☐ Ant	☐ Gradle	
☐ Pipeline	☐ GitHub Branch Source	☐ Pipeline: GitHub Groovy Libraries	☐ Pipeline Graph View	
☐ Git	☐ SSH Build Agents	☐ Matrix Authorization Strategy	☐ LDAP	
☐ Email Extension	☐ Mailer	☐ Dark Theme		

- After then **create admin profile** by **filling all details** then **click on save and continue** after then **save and finish** after then **click on start using Jenkin**.

Getting Started

Instance Configuration

Jenkins URL:

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

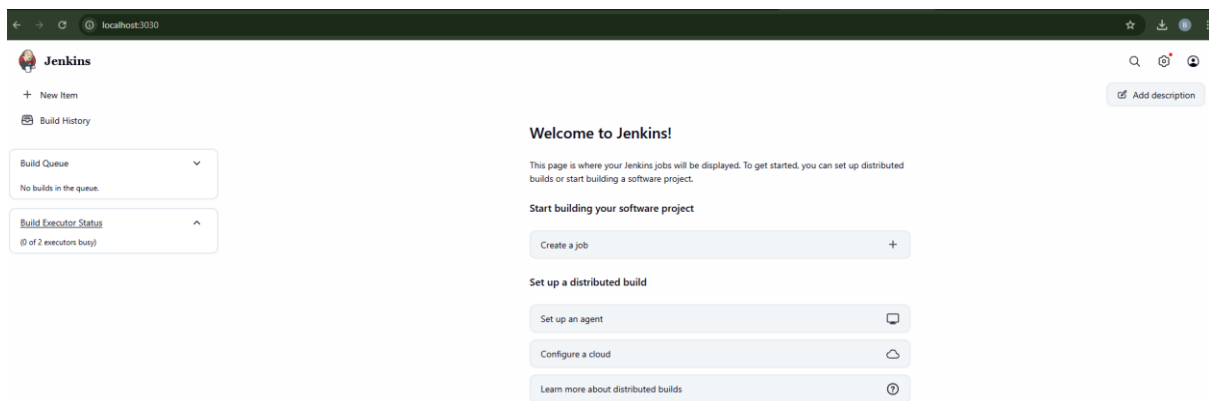
Getting Started

Jenkins is ready!

Your Jenkins setup is complete.

Start using Jenkins

- We can access it in the browser at **`http://localhost:8080`** or **`http://localhost:3030`**.



How to reset new username and password for Jenkins

1. Cmd : net stop Jenkins
2. Open c drive → program files → click on views → check mark hidden items → close window
3. Open c drive → program data → Jenkins → .jenkins folder should be deleted
4. Restart the PC
5. CMD: net start Jenkins
6. Open <https://localhost:3030>
7. Unlock Jenkins page will open then same procedure has to be followed

Lab6

Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests

Step 1: Login with user name and password

Step 2: Manage Jenkins-Tools-select maven installation->name->MAVEN_HOME->deselect the install automatically->copy the path of maven in your system and paste and save.

Step 3: Go to plugins->available plugins->search for maven integration->maven integration 3.2X and click on install.

Step 4: dashboard->new item->program6->click on freestyle project and ok.

Step 5: build set up->click on add build set up->select invoke top level maven targets->maven version select maven_home

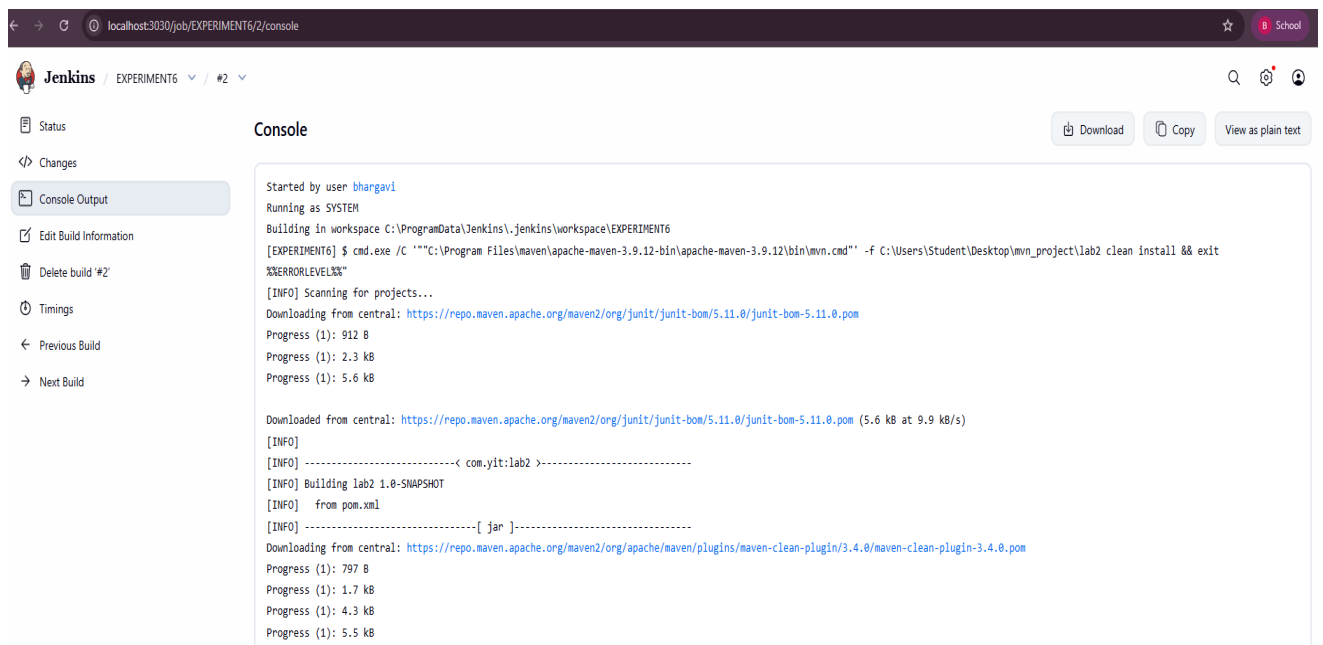
Goal->type the following

Clean install

Click on advance->pom->copy the path of pom.xml of program2 and paste, save.

Step 6: build now

Step 7: goto console and check the result



The screenshot shows the Jenkins web interface for a build named 'EXPERIMENT6' (build #2). The 'Console' tab is selected, displaying the build log. The log shows the build was started by user 'bhargavi' and is running as 'SYSTEM'. It details the execution of 'mvn clean install' in a workspace. The log includes progress bars for downloading dependencies from the central repository, such as 'junit-junit-bom-5.11.0' and 'maven-clean-plugin-3.4.0'. The build is currently in the 'Building' phase, with progress shown for 'Building lab2 1.0-SNAPSHOT'.

```
Started by user bhargavi
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\EXPERIMENT6
[EXPERIMENT6] $ cmd.exe /C ""C:\Program Files\maven\apache-maven-3.9.12\bin\apache-maven-3.9.12\bin\mvn.cmd"" -f C:\Users\Student\Desktop\mvn_project\lab2 clean install && exit
%ERRORLEVEL%
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/junit/junit-bom/5.11.0/junit-bom-5.11.0.pom
Progress (1): 912 B
Progress (1): 2.3 kB
Progress (1): 5.6 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/junit-bom/5.11.0/junit-bom-5.11.0.pom (5.6 kB at 9.9 kB/s)
[INFO]
[INFO] -----< com.yit:lab2 >-----
[INFO] Building lab2 1.0-SNAPSHOT
[INFO]    from pom.xml
[INFO] -----[ jar ]-----
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plugin/3.4.0/maven-clean-plugin-3.4.0.pom
Progress (1): 797 B
Progress (1): 1.7 kB
Progress (1): 4.3 kB
Progress (1): 5.5 kB
```

```
Jenkins / EXPERIMENT6 / #2

Progress (1): 94/157 kB
Progress (1): 111/157 kB
Progress (1): 127/157 kB
Progress (2): 127/157 kB | 7.7/196 kB
Progress (2): 143/157 kB | 7.7/196 kB
Progress (2): 143/157 kB | 16/196 kB
Progress (2): 157 kB | 16/196 kB
Progress (2): 157 kB | 32/196 kB
Progress (2): 157 kB | 48/196 kB
Progress (2): 157 kB | 65/196 kB
Progress (2): 157 kB | 81/196 kB
Progress (2): 157 kB | 98/196 kB
Progress (2): 157 kB | 114/196 kB
Progress (2): 157 kB | 130/196 kB
Progress (2): 157 kB | 147/196 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-api/1.9.18/maven-resolver-api-1.9.18.jar (157 kB at 1.6 MB/s)
Progress (1): 163/196 kB
Progress (1): 179/196 kB
Progress (1): 196 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-util/1.9.18/maven-resolver-util-1.9.18.jar (196 kB at 899 kB/s)
[INFO] Installing C:\Users\Student\Desktop\mvn_project\lab2\pom.xml to C:\Windows\system32\config\systemprofile\.m2\repository\com\yit\lab2\1.0-SNAPSHOT\lab2-1.0-SNAPSHOT.pom
[INFO] Installing C:\Users\Student\Desktop\mvn_project\lab2\target\lab2-1.0-SNAPSHOT.jar to C:\Windows\system32\config\systemprofile\.m2\repository\com\yit\lab2\1.0-SNAPSHOT\lab2-1.0-SNAPSHOT.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 26.738 s
[INFO] Finished at: 2026-03-10T11:43:27+05:30
[INFO] -----
Finished: SUCCESS
```

To test

Step 1: go to build history click on configure change the goal to test and save build again

```
localhost:3030/job/EXPERIMENT6/3/console

Jenkins / EXPERIMENT6 / #3

Status
Changes
Console Output
Edit Build Information
Delete build #3
Timings
Previous Build

Console

Started by user bhargavi
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\jenkins\workspace\EXPERIMENT6
[EXPERIMENT6] $ cmd.exe /C ""C:\Program Files\maven\apache-maven-3.9.12\bin\mvn.cmd"" -f C:\Users\Student\Desktop\mvn_project\lab2 test && exit %ERRORLEVEL%
[INFO] Scanning for projects...
[INFO] -----< com.yit:lab2 >-----
[INFO] Building lab2 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- resources:3.3.1:resources (default-resources) @ lab2 ---
[INFO] skip non existing resourceDirectory C:\Users\Student\Desktop\mvn_project\lab2\src\main\resources
[INFO] --- compiler:3.13.0:compile (default-compile) @ lab2 ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- resources:3.3.1:testResources (default-testResources) @ lab2 ---
[INFO] skip non existing resourceDirectory C:\Users\Student\Desktop\mvn_project\lab2\src\test\resources
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ lab2 ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.3.0:test (default-test) @ lab2 ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnit4PlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.yit.demo.AppTest
Running test: 2 + 3 = 5
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.050 s -- in com.yit.demo.AppTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.407 s
[INFO] Finished at: 2026-03-10T11:45:40+05:30
[INFO] -----
Finished: SUCCESS
```


Lab-7

Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook.

Theory

Ansible is an open-source automation tool used for: Configuration management, Application deployment and Task automation.

It works in an agentless manner (no software required on client machines), using SSH.

Inventory

The inventory file in Ansible contains the list of hosts or network devices that you want to manage. It is usually located at `/etc/ansible/hosts` by default.

To define hosts in the inventory file, open it in a text editor and add the IP addresses or domain names of your servers. You can organize hosts into groups and assign variables to them.

Example inventory file:

[web]

```
webserver1 ansible_host=192.168.0.10 ansible_user=admin
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

[database]

```
dbserver1 ansible_host=192.168.0.20 ansible_user=admin
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

Explanation:

- Groups like webserver and dbserver help organize machines.
- Ansible uses this file to know where to execute tasks.

Playbooks

Ansible playbooks are YAML files that describe the desired state of your system. Playbooks consist of plays, which are sets of tasks executed on a specific group of hosts.

A simple playbook example:

- name: Install and start Apache

hosts: web

become: true

tasks:

- name: Install Apache package

apt:

name: apache2

state: present

- name: Start Apache service

service:

name: apache2

state: started

enabled: true

Save the above YAML code to a file named webserver.yml.

Explanation:

- hosts: target machines
- tasks: list of actions
- become: runs tasks as root (sudo)

Modules

Modules are reusable units that perform specific tasks.

Common Modules:

- yum / apt → install packages
- service → manage services
- copy → copy files
- file → manage file properties

Example:

- name: Start Apache

service:

name: httpd

state: started

Automating Server Configurations with Playbooks

Ansible playbooks automate repetitive server tasks such as:

- Installing software
- Configuring services
- Managing users
- Updating systems

Workflow:

1. Define inventory (servers)
2. Write playbook (tasks)
3. Run playbook
4. Ansible connects via SSH and executes tasks

Advantages:

- Saves time
- Reduces human error
- Ensures consistency across servers

Ansible Control Node vs Managed Nodes

1. Ansible Control Node (Master Node)

The Control Node is the system where Ansible is installed and from which all automation tasks are executed.

Key Functions:

- Runs Ansible commands and playbooks
- Stores:
 - Inventory file
 - Playbooks (YAML files)
 - Configuration files
- Connects to managed nodes using SSH

2. Managed Nodes (Target Nodes / Hosts)

Managed Nodes are the remote systems that Ansible controls.

Key Functions:

- Execute tasks sent by the control node

- Perform operations like:
 - Installing software
 - Starting services
 - Updating configurations

Procedures

Step 1: Update your system

Open Terminal and run:

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Install Ansible

```
sudo apt install ansible -y
```

Check:

```
ansible --version
```

Step 3: Install SSH (needed for Ansible)

```
sudo apt install openssh-client -y
```

Step 4: Create your project folder

```
mkdir ansible_project
```

```
cd ansible_project
```

Step 5: Create Inventory file

```
nano inventory
```

Paste:

```
[local]
```

```
localhost ansible_connection=local
```

Save and exit.

Step 6: Test Ansible

```
ansible -i inventory all -m ping
```

Output should be:

```
pong
```

Step 7: Create Playbook

```
nano playbook.yml
```

Paste:

```
- name: Install nginx
  hosts: local
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
    - name: Start nginx
      service:
        name: nginx
        state: started
```

Step 8: Run Playbook

```
ansible-playbook -i inventory playbook.yml
```

Step 9: Verify

```
curl http://localhost
```

If you see HTML → SUCCESS 🎉

LAB-8

Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins.

- Compiled maven project should be pushed to github profile.
- Jenkins CI Pipeline (build Maven project)
- Artifact generation (.jar/.war)
- Ansible deployment from Ubuntu

PART 1: Jenkins Setup (CI Pipeline)

1. Install Required Plugins in Jenkins

Go to:

Manage Jenkins → Plugins → Available Plugins

Install:

- Git Plugin
- Maven Integration Plugin
- Pipeline Plugin --. pipeline stage view
- SSH Agent Plugin

Restart Jenkins after install.

2. Install git in ubuntu terminal

```
sudo apt update
sudo apt install git -y
which git
/usr/bin/git
```

a. Configure Git in Jenkins

Go to:

Manage Jenkins → Global Tool Configuration

- Scroll to **Git**
- Click **Add Git**
- Name: Default
- Path to Git executable: `/usr/bin/git` (or your path)

```
sudo systemctl restart jenkins
```

3. Configure Tools in Jenkins

Go to:

Manage Jenkins → Global Tool Configuration

Add Maven

- Name: Maven
- Select: Install automatically

Add JDK

- Name: JDK
- Install automatically

3. Create Jenkins Pipeline Job

Go to:

New Item → Pipeline → Name: Maven-CI-CD → OK

4. Configure GitHub Repository

Inside job:

- Go to **Pipeline → Definition**
- Select: Pipeline script
- below is the jenkins file

```
pipeline {
    agent any

    tools {
        maven 'Maven'
        jdk 'JDK'
    }

    stages {

        stage('Clean') {
            steps {
                cleanWs()
            }
        }

        stage("shell script"){
            steps{
                sh 'whoami'
            }
        }

        stage('clone repository and Checkout') {
            steps {
                git branch: 'main', url:
'https://github.com/bhargavi-'
```

```

        }
    }

    stage('Build') {
        steps {
            sh 'mvn clean install'
        }
    }

    stage('Copy Artifact') {
        steps {
            sh 'cp target//lab2-1.0-SNAPSHOT.jar
/home/lab/deploy/'
        }
    }

    stage('Run Ansible Playbook') {
        steps {
            sh 'ansible-playbook /home/lab/deploy/deploy.yml -
i /home/lab/deploy/hosts'
        }
    }
}

```

6. Build Job

Click:

Build Now

Check Console Output → Ensure:

- Maven build success
- .jar created in target/

PART 2: Ubuntu (Ansible Setup)

1. Install Ansible

On Ubuntu:

sudo apt update

sudo apt install ansible -y

ansible --version

check present working directory:

pwd

o/p: /home/lab

2. Create Project Structure

```
mkdir -p ~/deploy
cd ~/deploy
```

3. Create Inventory File (inside deploy directory)

```
nano hosts
[app]
localhost ansible_connection=local
```

4. Create Ansible Playbook (inside deploy directory)

```
nano deploy.yml
- name: Deploy Maven Artifact Locally
  hosts: localhost
  connection: local

vars:
  deploy_dir: /home/lab/deploy/myapp # Where the app will run
  source_dir: /home/lab/deploy/      # Where Jenkins copied the JAR

tasks:

  - name: Create deployment directory
    file:
      path: "{{ deploy_dir }}"
      state: directory

  - name: Stop existing app if running
    shell: pkill -f 'java -jar'
    register: pkill_result
    ignore_errors: yes
    changed_when: pkill_result.rc == 0

  - name: Copy JAR file to deployment directory
    copy:
      src: "/home/lab/deploy/lab2-1.0--SNAPSHOT.jar"
      dest: "{{ deploy_dir }}/lab2-1.0-SNAPSHOT.jar"
      remote_src: yes

  - name: Run new JAR
    shell: "nohup java -jar {{ deploy_dir }}/*.jar > {{ deploy_dir }}/app.log 2>&1 &"
```

```
lab@HP-PRODESK: ~/deploy
GNU nano 7.2 deploy.yml
name: Deploy Maven Artifact Locally
hosts: localhost
connection: local

vars:
  deploy_dir: /home/lab/deploy/myapp # Where the app will run
  source_dir: /home/lab/deploy/      # Where Jenkins copied the JAR

tasks:
  - name: Create deployment directory
    file:
      path: "{{ deploy_dir }}"
      state: directory

  - name: Stop existing app if running
    shell: pkill -f 'java -jar'
    register: pkill_result
    ignore_errors: yes
    changed_when: pkill_result.rc == 0

  - name: Copy JAR file to deployment directory
    copy:
      src: "/home/lab/deploy/lab2-1.0-SNAPSHOT.jar"
      dest: "{{ deploy_dir }}/lab2-1.0-SNAPSHOT.jar"
      remote_src: yes
```

5. Give Permissions

`chmod +x deploy.yml`

a. NOW go to jenkins pipeline script add the blocks “copy artifact” and “run ansible playbook” in pipeline script

1. Jenkins builds project (mvn clean package)
2. JAR gets created
3. Jenkins calls Ansible
4. Ansible deploys JAR

b. In ubuntu Terminal

`sudo chown -R jenkins:jenkins /home/lab/deploy`

`sudo chmod -R 777 /home/lab/deploy`

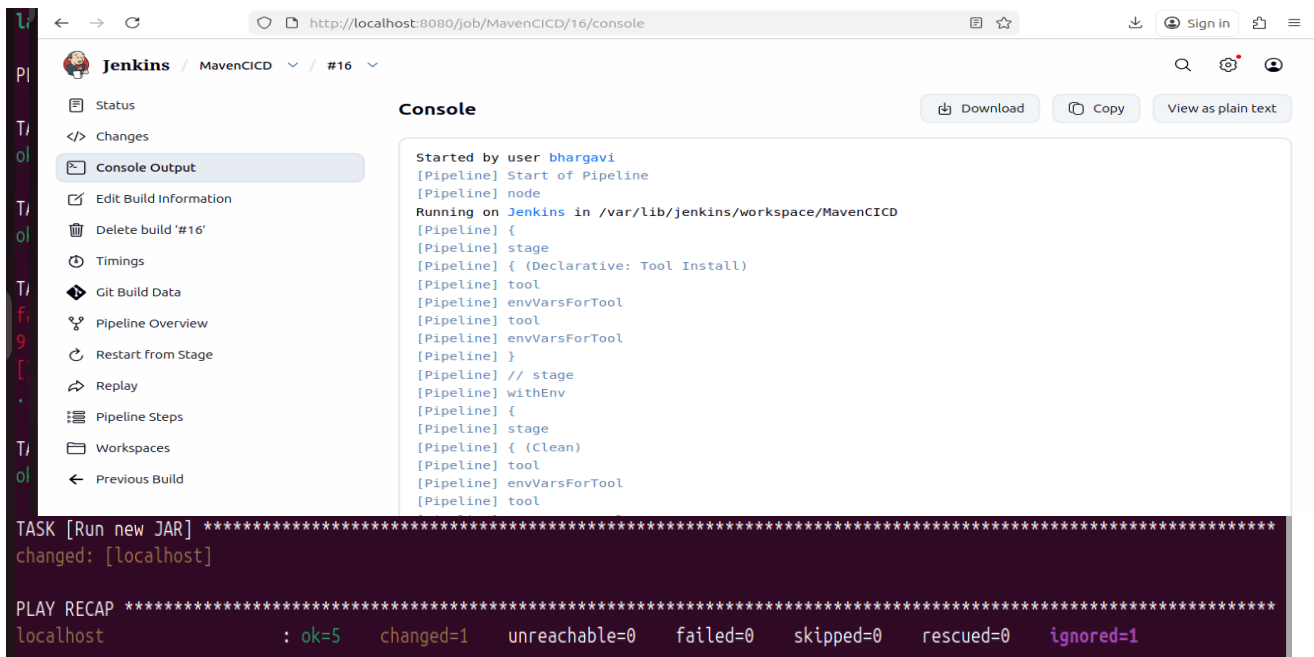
`sudo usermod -aG lab jenkins`

`sudo systemctl restart Jenkins`

c. go to Jenkins job and give “BUILD NOW”

6. Test Playbook Manually

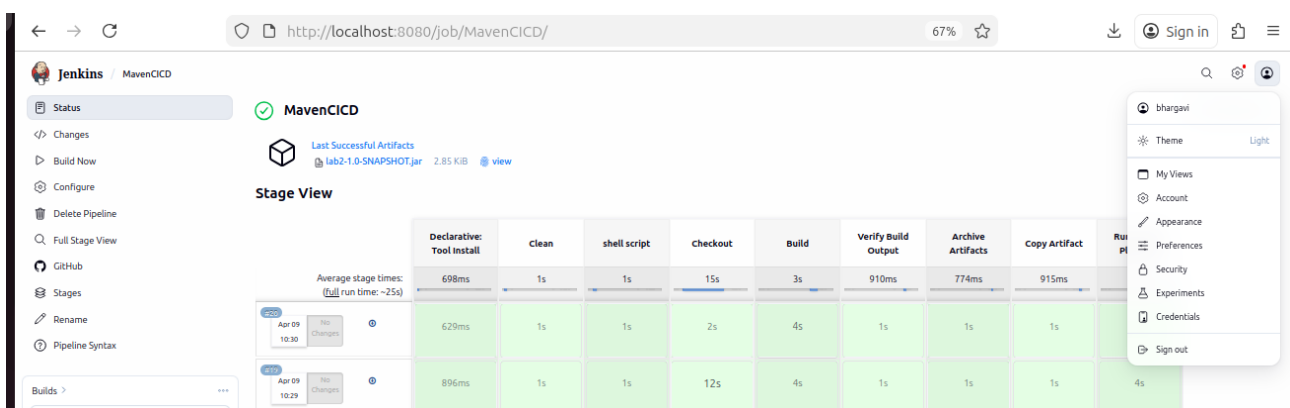
`ansible-playbook -i hosts deploy.yml`



go back to jenkins job Maven-CI-CD and click on build now
 artifact output

```
lab@HP-PRODESK:~/deploy$ ls
deploy.yml  hosts  lab2-1.0-SNAPSHOT.jar
```

pipeline stage view output

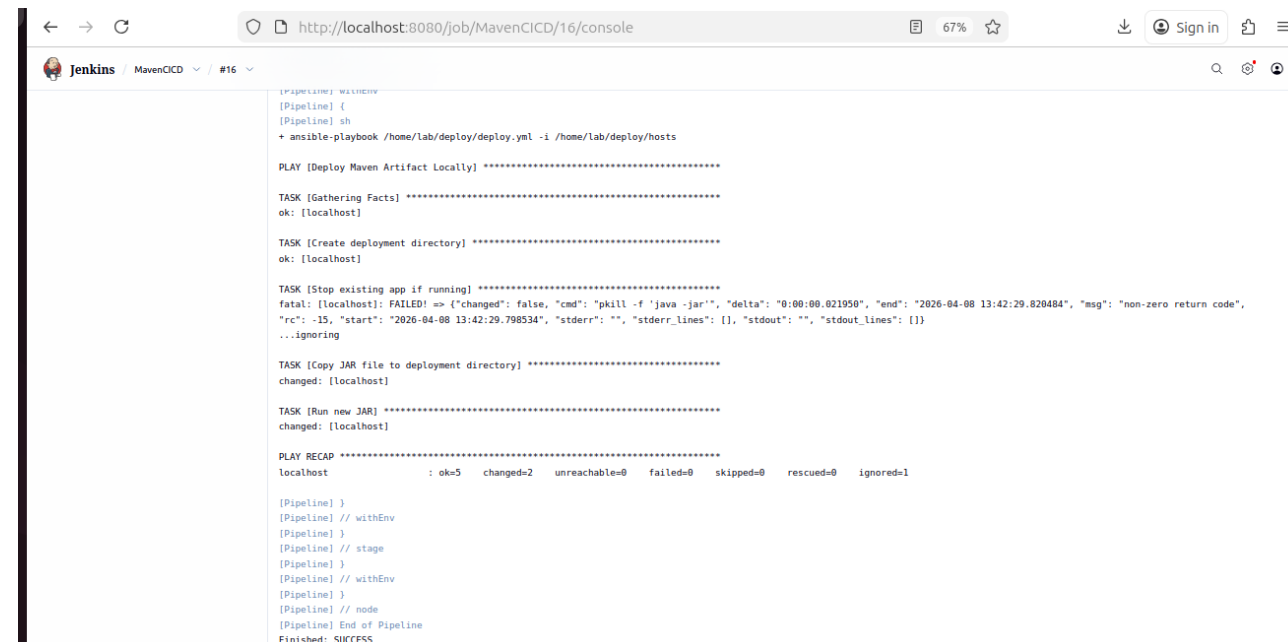


pipeline console output



The screenshot shows the Jenkins console output for a build named 'MavenCICD' with build number '#16'. The output is a log of Maven commands and their results. It starts with 'Running test: 2 + 3 = 5', followed by 'Building jar: /var/lib/jenkins/workspace/MavenCICD/target/lab2-1.0-SNAPSHOT.jar', and then 'Installing /var/lib/jenkins/workspace/MavenCICD/pom.xml to /var/lib/jenkins/.m2/repository/com/yit/lab2/1.0-SNAPSHOT/lab2-1.0-SNAPSHOT.pom'. The build concludes with 'BUILD SUCCESS' and a total time of 2.881 s.

```
[[1:34mINFO[m]
[[1:34mINFO[m] .....
[[1:34mINFO[m] T E S T S
[[1:34mINFO[m] .....
[[1:34mINFO[m] Running com.yit.demo.[1mAppTest[m
Running test: 2 + 3 = 5
[[1:34mINFO[m] [1:32mTests run: 0;1:32m[m, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.065 s -- in com.yit.demo.[1mAppTest[m
[[1:34mINFO[m]
[[1:34mINFO[m] Results:
[[1:34mINFO[m]
[[1:34mINFO[m] [1:32mTests run: 1, Failures: 0, Errors: 0, Skipped: 0[m
[[1:34mINFO[m]
[[1:34mINFO[m]
[[1:34mINFO[m] [1m--- [0:32mmaven-jar-plugin:3.4.2:jar[m [1mdefault-jar][m @ [36mlab2[0:1m ---[m
[[1:34mINFO[m] Building jar: /var/lib/jenkins/workspace/MavenCICD/target/lab2-1.0-SNAPSHOT.jar
[[1:34mINFO[m]
[[1:34mINFO[m] [1m--- [0:32mmaven-install-plugin:3.1.2:install[m [1mdefault-install][m @ [36mlab2[0:1m ---[m
[[1:34mINFO[m] Installing /var/lib/jenkins/workspace/MavenCICD/pom.xml to /var/lib/jenkins/.m2/repository/com/yit/lab2/1.0-SNAPSHOT/lab2-1.0-SNAPSHOT.pom
[[1:34mINFO[m] Installing /var/lib/jenkins/workspace/MavenCICD/target/lab2-1.0-SNAPSHOT.jar to /var/lib/jenkins/.m2/repository/com/yit/lab2/1.0-SNAPSHOT/lab2-1.0-SNAPSHOT.jar
[[1:34mINFO[m] [1m-----[m
[[1:34mINFO[m] [1:32mBUILD SUCCESS[m
[[1:34mINFO[m] [1m-----[m
[[1:34mINFO[m] Total time: 2.881 s
[[1:34mINFO[m] Finished at: 2026-04-08T13:42:24+05:30
[[1:34mINFO[m] [1m-----[m
[Pipeline] }
[Pipeline] // withEnv
```



The screenshot shows the Jenkins console output for the same build, displaying the Ansible playbook execution. The output includes the playbook name 'deploy.yml', the tasks 'Gathering Facts', 'Create deployment directory', 'Stop existing app if running', 'Copy JAR file to deployment directory', and 'Run new JAR'. The final status is 'PLAY RECAP' showing 5 OK, 2 changed, and 0 failed tasks. The build ends with 'Finished: SUCCESS'.

```
[Pipeline] WALHEDEV
[Pipeline] {
[Pipeline] sh
+ ansible-playbook /home/lab/deploy/deploy.yml -i /home/lab/deploy/hosts

PLAY [Deploy Maven Artifact Locally] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Create deployment directory] *****
ok: [localhost]

TASK [Stop existing app if running] *****
fatal: [localhost]: FAILED! => {"changed": false, "cmd": "pkill -f 'java -jar'", "delta": "0:00:00.021950", "end": "2026-04-08 13:42:29.020484", "msg": "non-zero return code", "rc": -15, "start": "2026-04-08 13:42:29.798534", "stderr": "", "stderr_lines": [], "stdout": "", "stdout_lines": []}
...ignoring

TASK [Copy JAR file to deployment directory] *****
changed: [localhost]

TASK [Run new JAR] *****
changed: [localhost]

PLAY RECAP *****
localhost      : ok=5  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=1

[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

PART 3: Full Flow (What Happens)

1. Jenkins pulls code from GitHub
2. Maven builds project (mvn clean install)
3. .jar file created in target/
4. Jenkins copies .jar → Ubuntu (Ansible server)
5. Jenkins triggers Ansible
6. Ansible:
 - Copies file to /home/lab/deploy/myapp
 - Stops old app and Starts new app

LAB 10: Creating Build Pipelines: Building a Maven/Gradle Project with Azure Pipelines, Integrating Code Repositories (e.g., GitHub, Azure Repos), Running Unit Tests and Generating Reports.

Microsoft

bhargavi k rao Sign out



bhargavi k rao [Edit profile](#)

bhargavirao954@gmail.com

Microsoft account

India

bhargavirao954@gmail.com

Visual Studio Dev Essentials

Get everything you need to build and deploy your app on any platform.

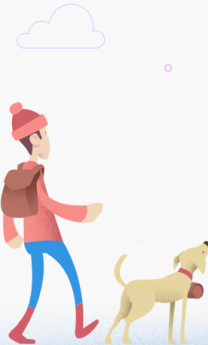
[Use your benefits](#)



Get started with Azure DevOps

Plan better, code together, ship faster with Azure DevOps

[Create new organization](#)



 Azure DevOps

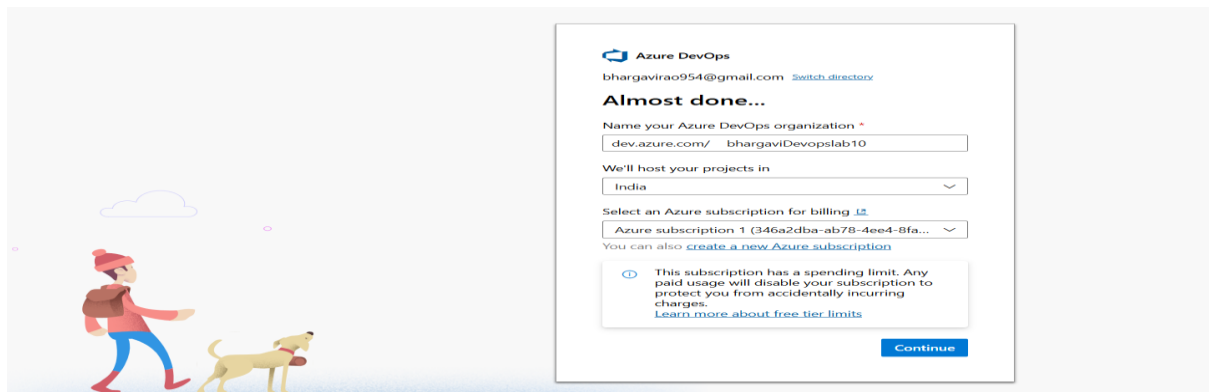
bhargavirao954@gmail.com [Switch directory](#)

Get started with Azure DevOps

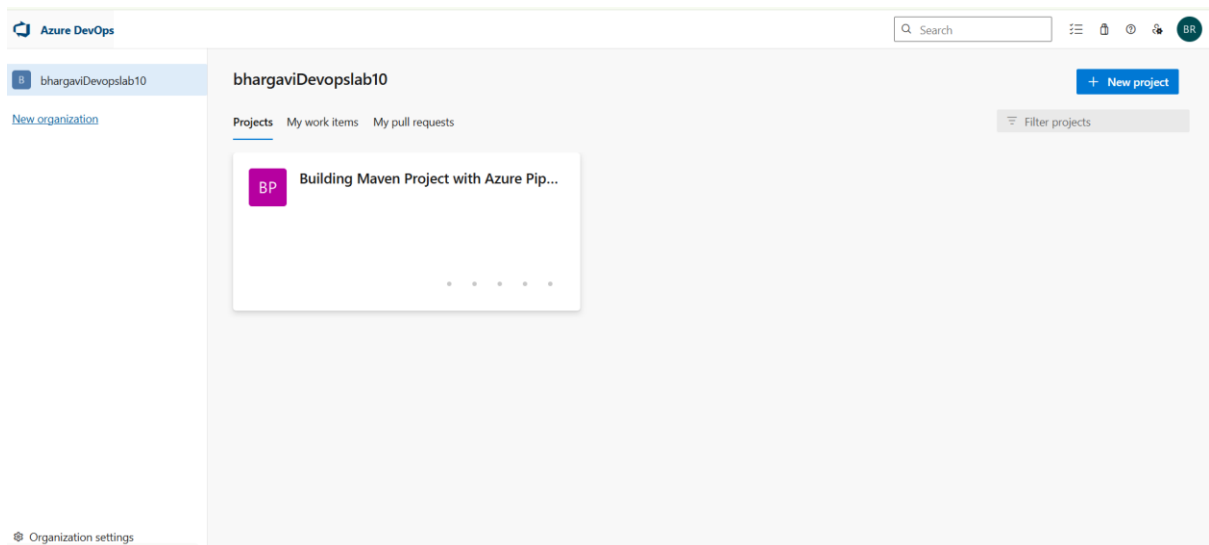
Choosing **Continue** means that you agree to our [Terms of Service](#), [Privacy Statement](#), and [Code of Conduct](#).

☐ I would like information, tips, and offers about Azure DevOps and other Microsoft products and services. [Privacy Statement](#).

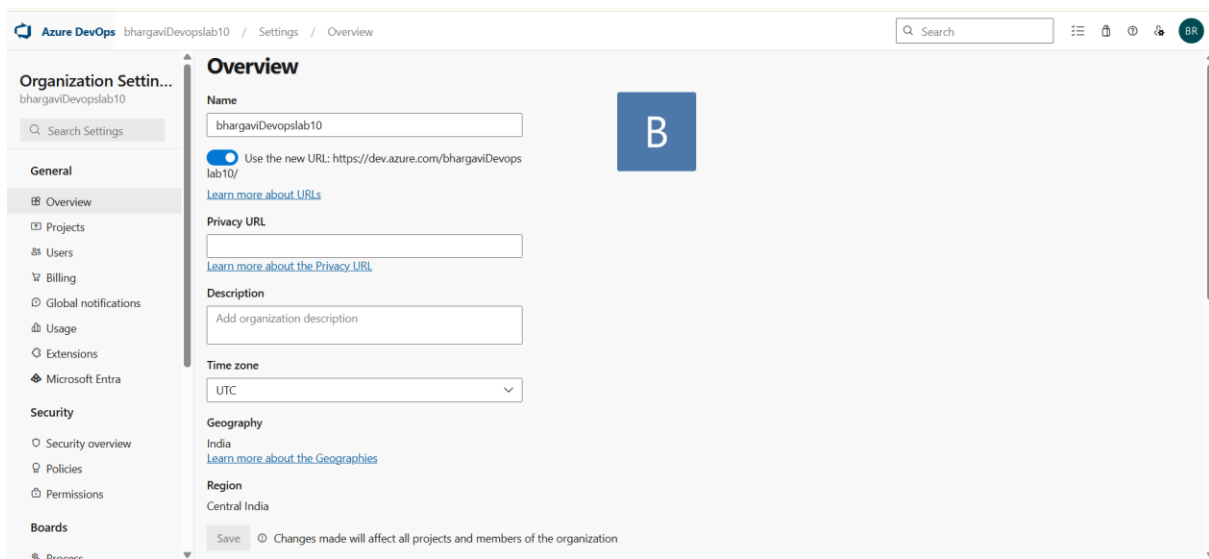
[Continue](#)



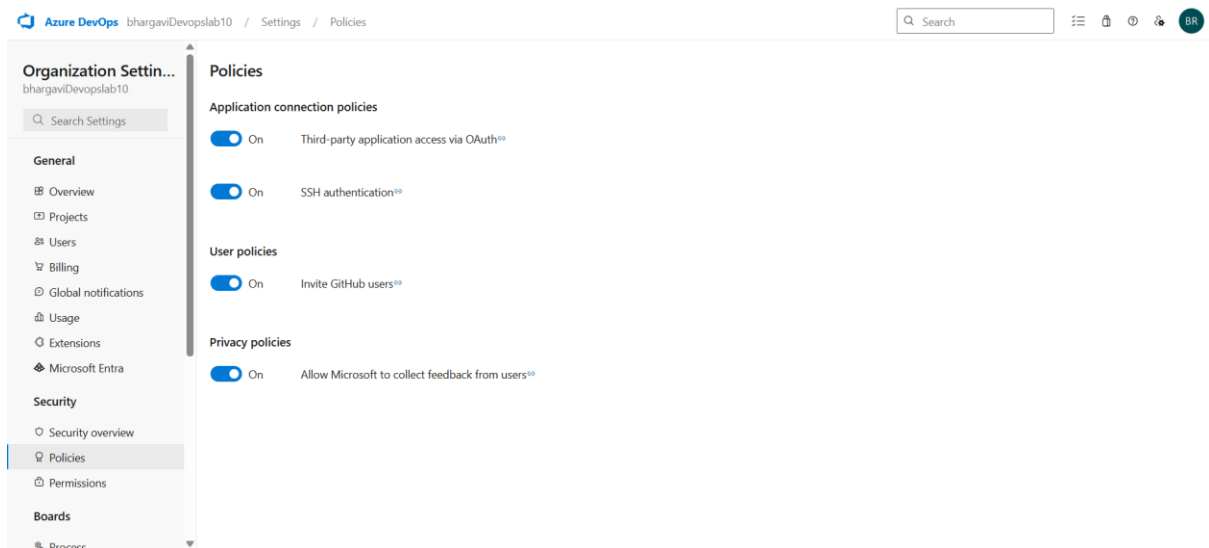
Step 1: Go to your organization and click on Organization settings



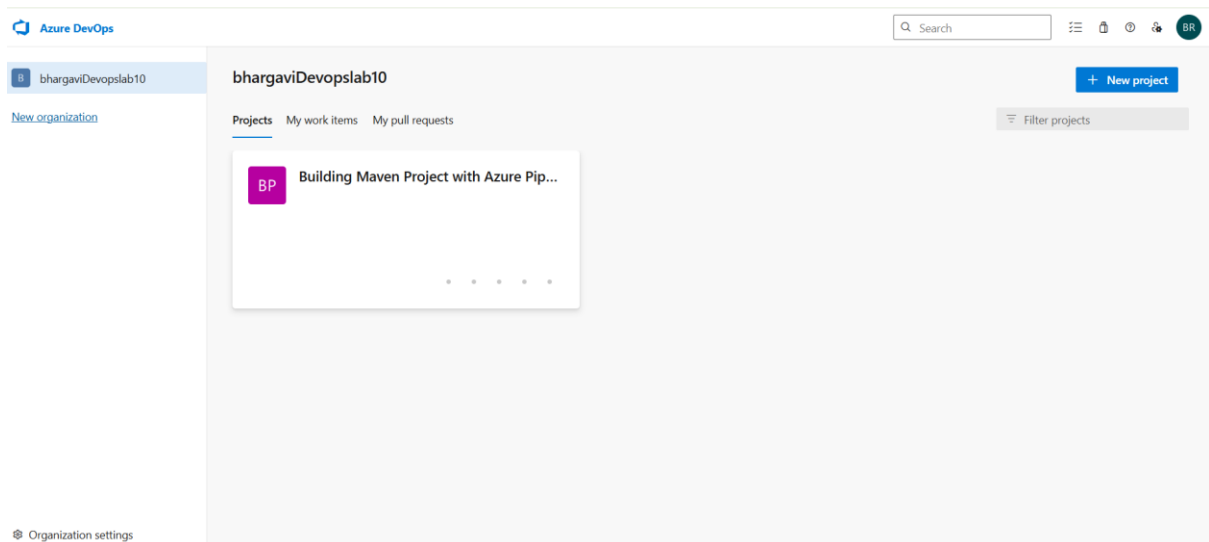
Step 2: Organization settings click on policies



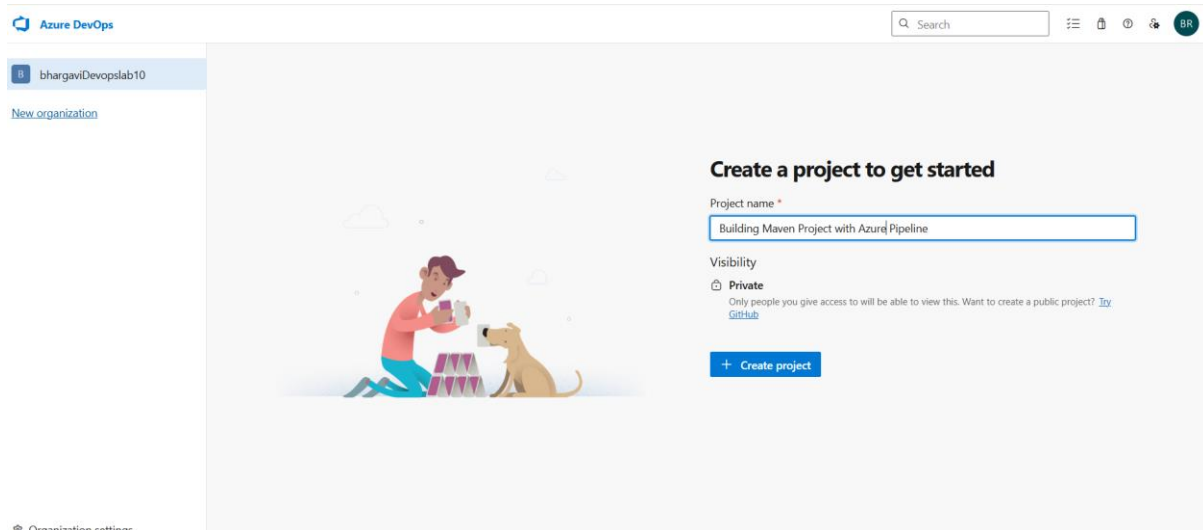
Step 3: On the Invite GitHub User and third party application access via OAuth



Step 4: Go back to Organization and click on new project

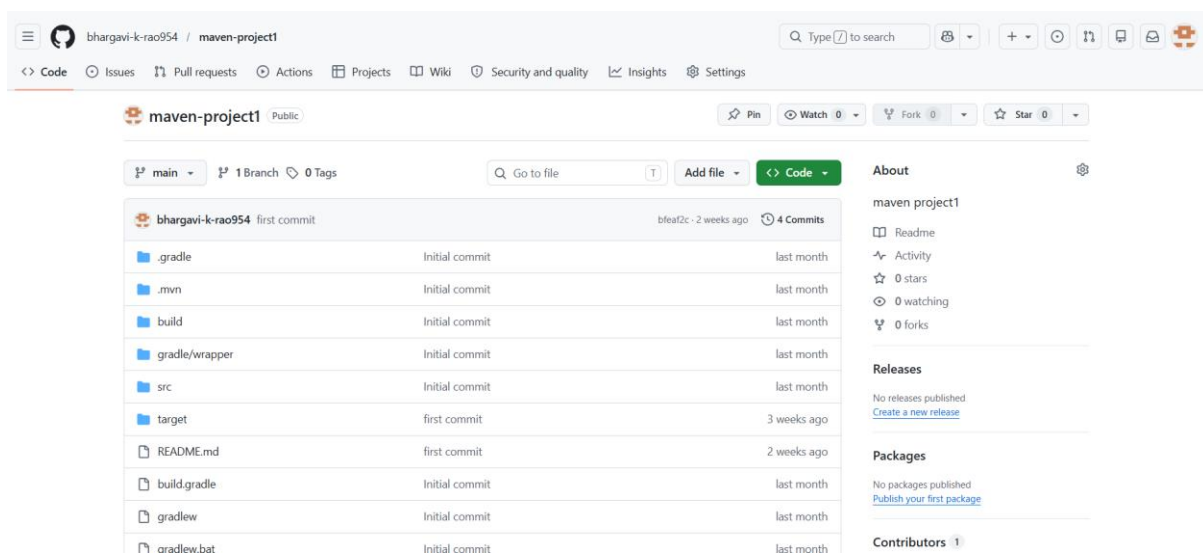


Step 5: Give the name for the Project and click on create project

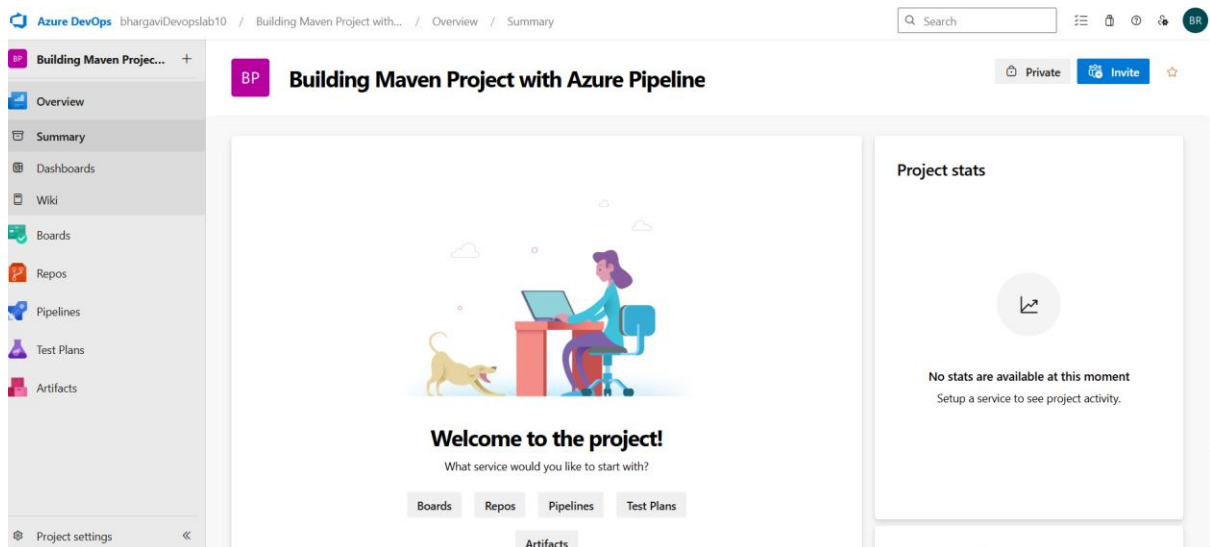


Step 6: Project Name: **Building Maven Project with Azure Pipeline**

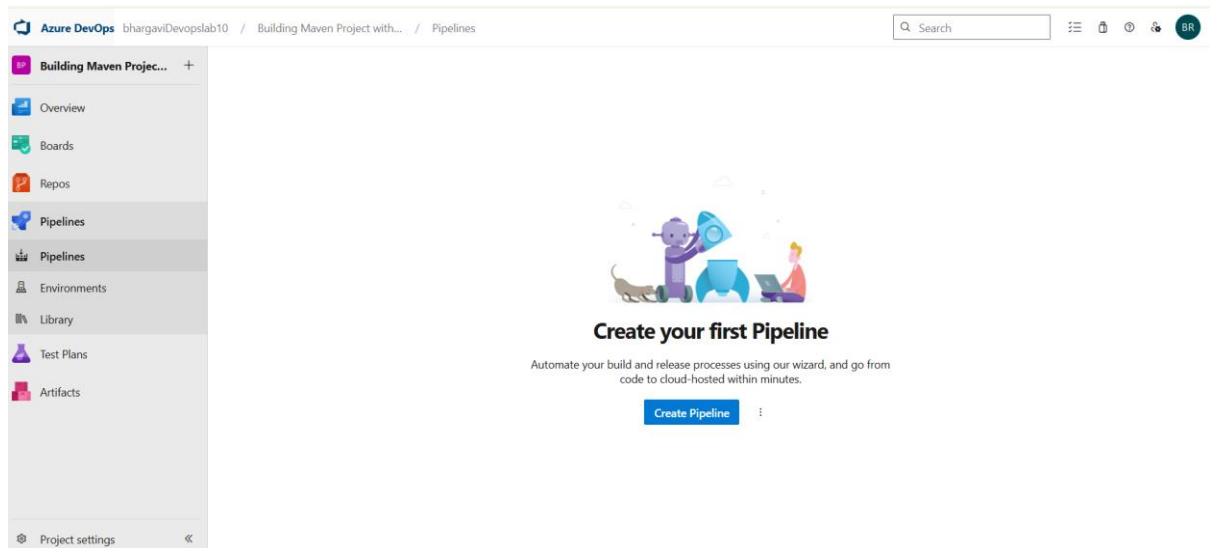
You need to create maven project as we followed steps in experiment 2, and push that project to GitHub as Maven Project Repo. (If already repo is present then you can use that Maven Project Repo). Login to your GitHub profile.



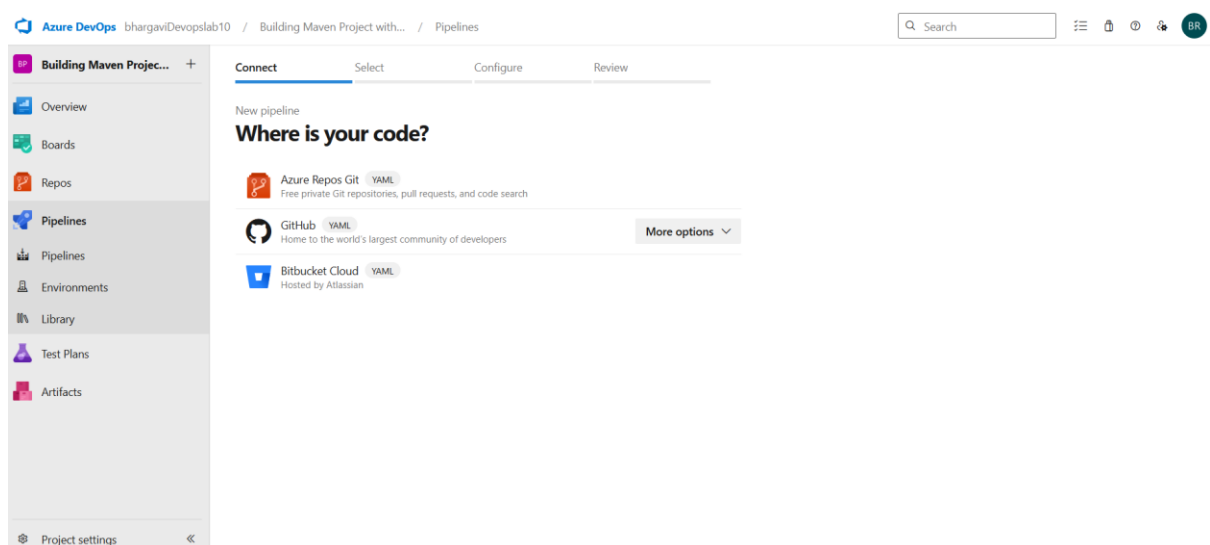
Step 7: Now goto Azure Devops Organization and click on the project created Building Maven Project with Azure Pipeline and inside the project click on pipeline.



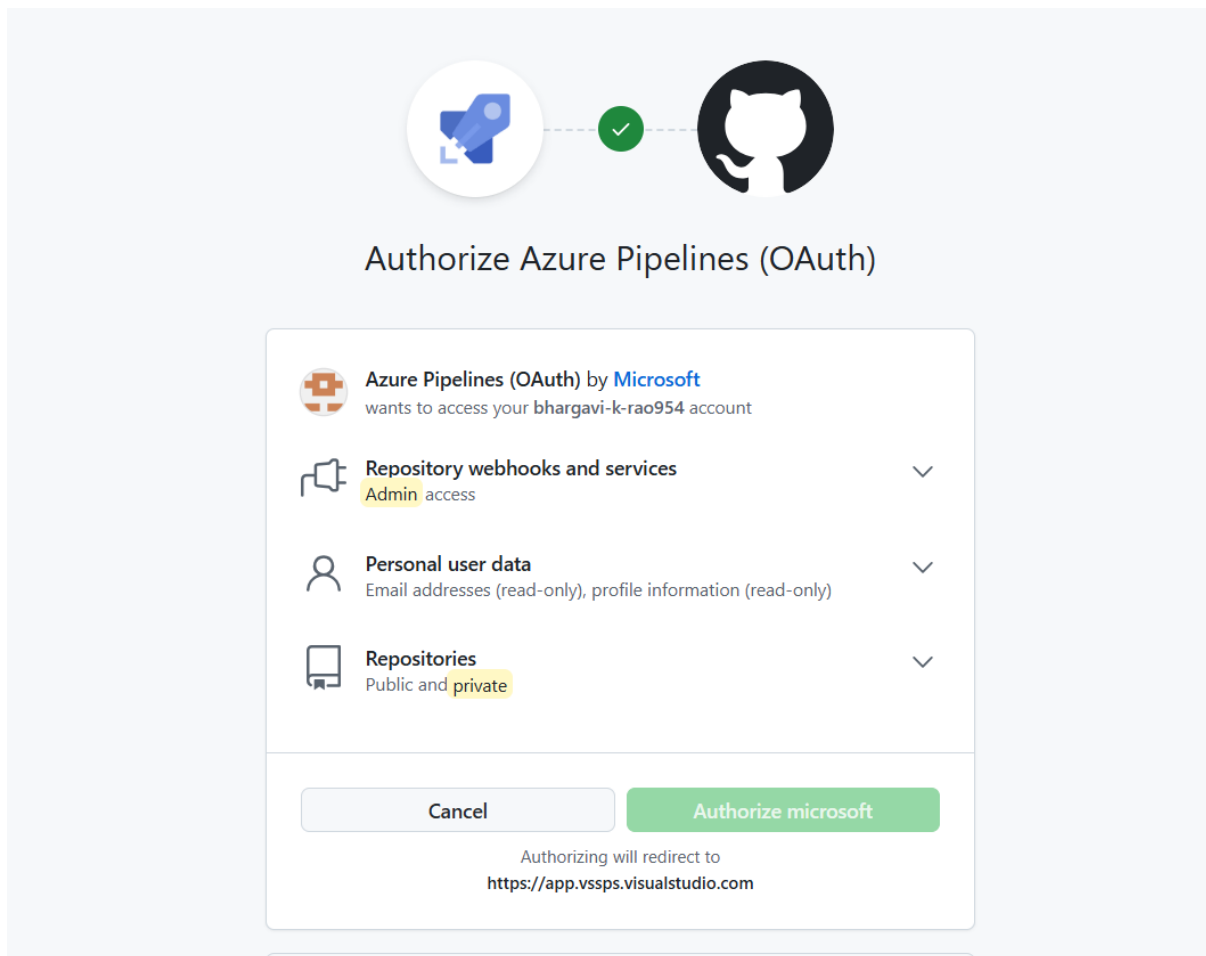
Step 8: Click on create pipeline



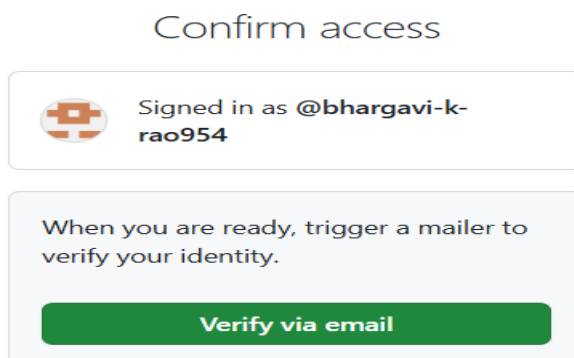
Step 9: After creating Pipeline select type of repo as Github



Step 10: When we select GitHub, we will this below window. In this select authorize Microsoft

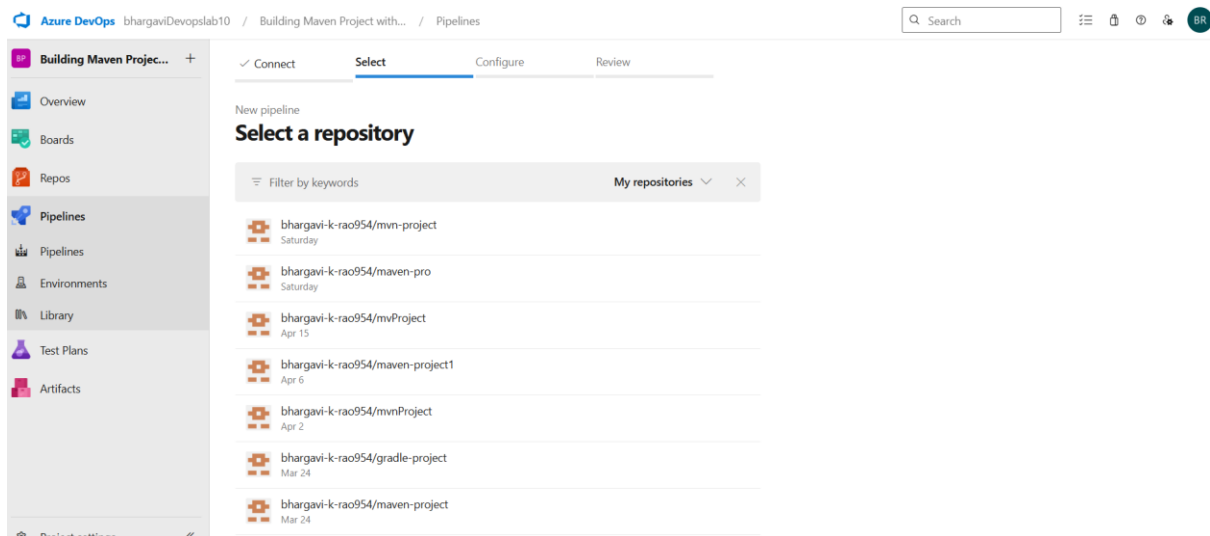


Click on verify email

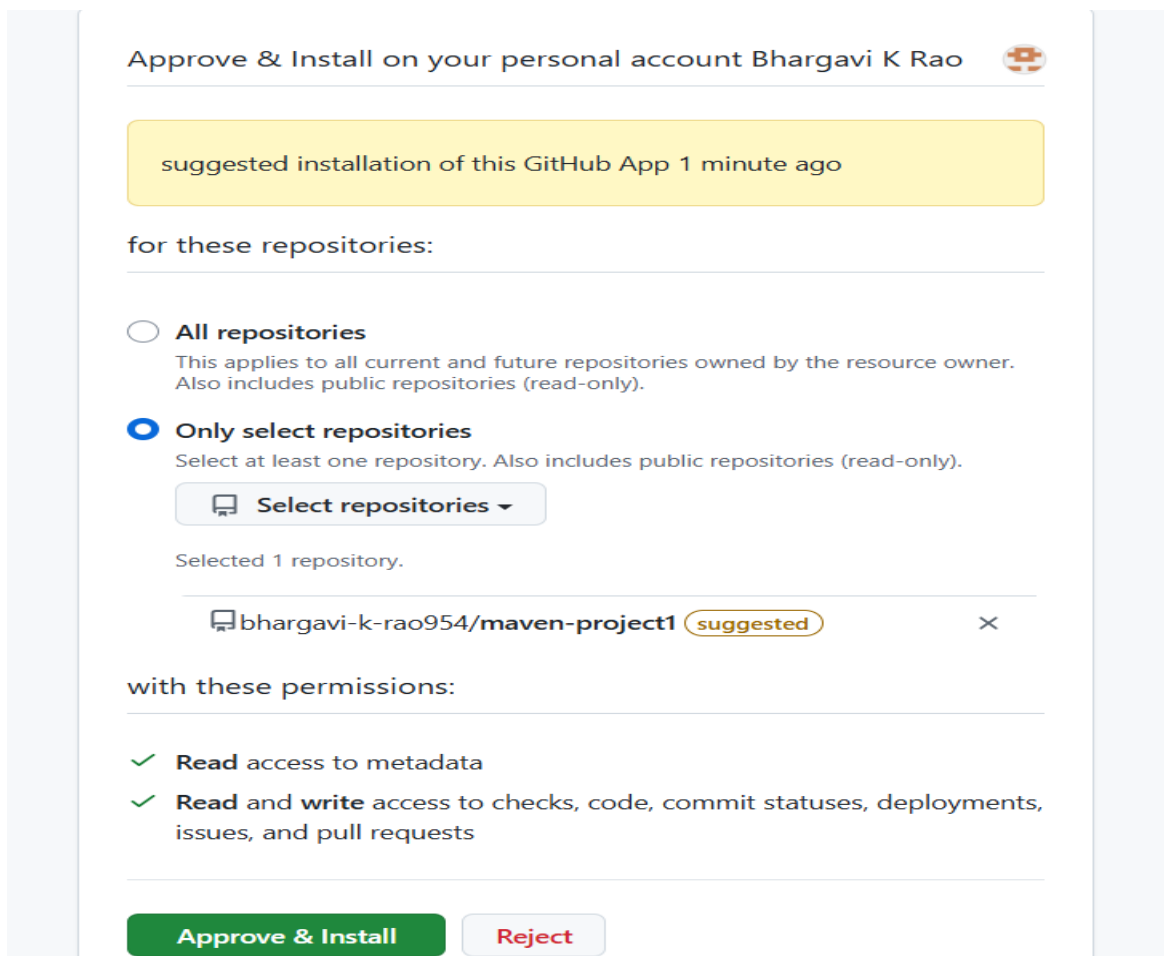


Tip: You are entering [sudo mode](#). After you've performed a sudo-protected action, you'll only be asked to re-authenticate again after a few hours of inactivity.

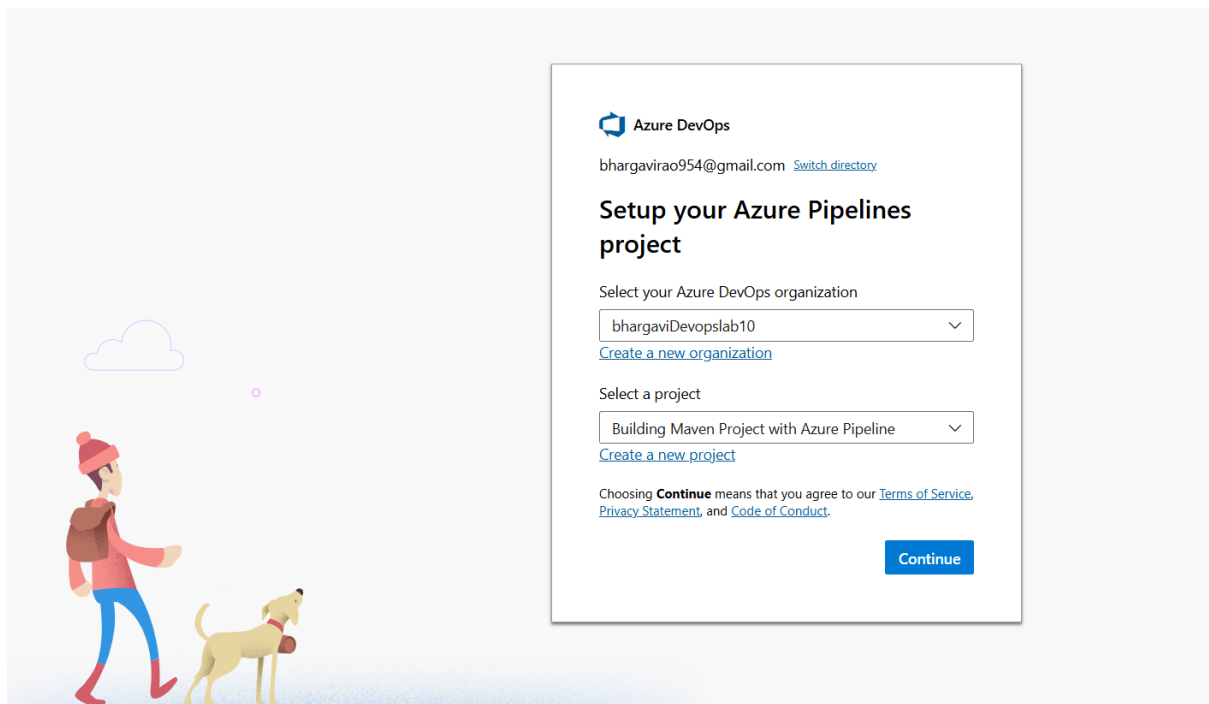
Step 11: We will get the code in the email (Gmail which is connected to the GitHub), enter that code now all the repos of GitHub we can see the azure window



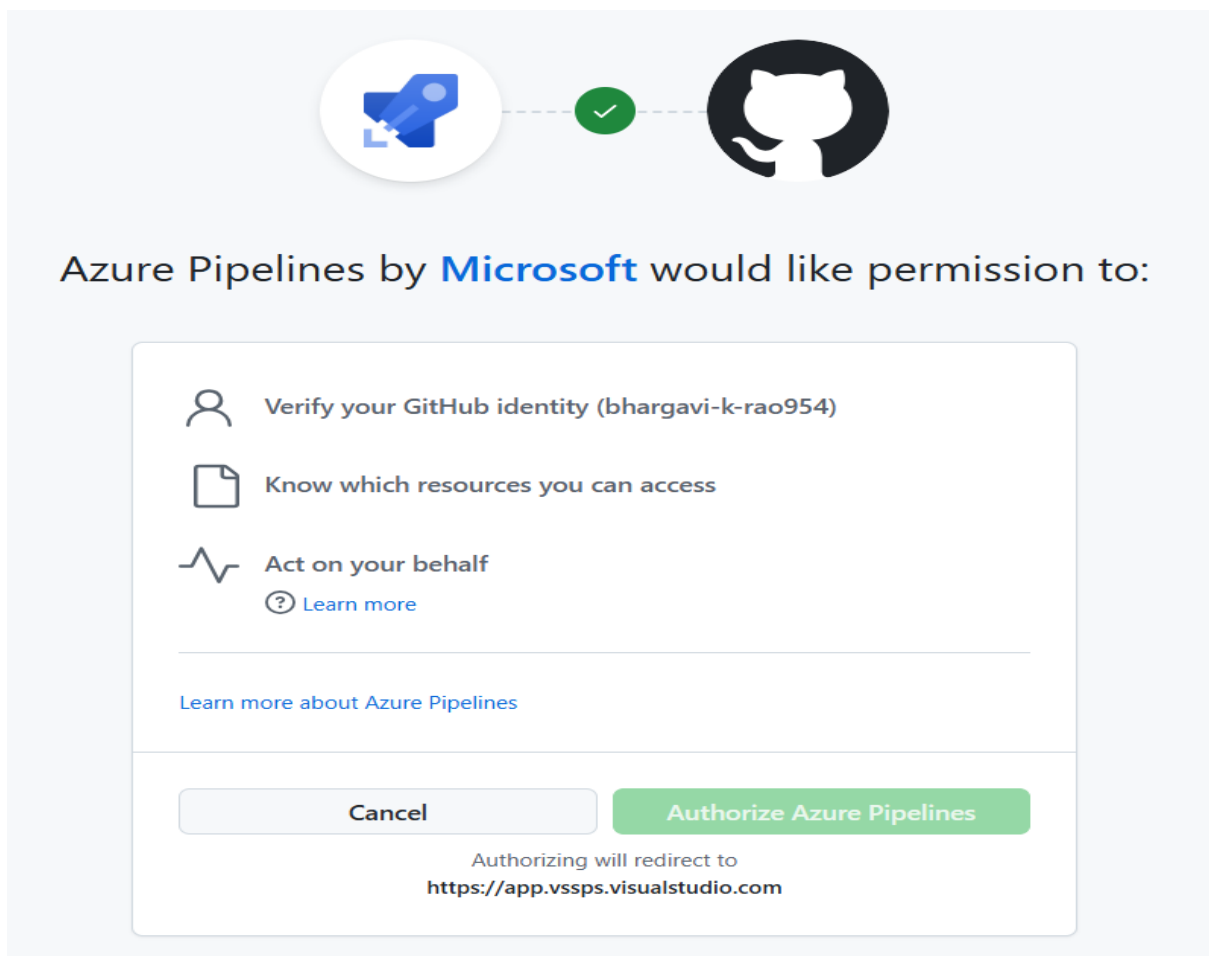
Step 12: We need to run the Maven project; in my case it is maven-project1 and I will select that repo and run. If selected repo is not public, we will get the below option and click on Approve & install. Later we need verify Microsoft account.

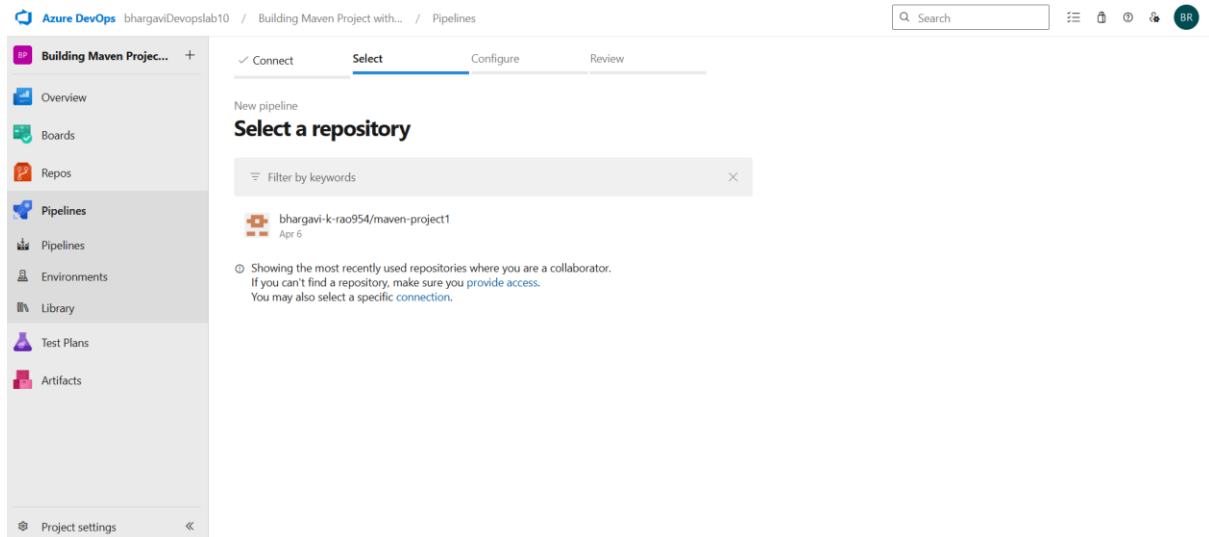


Step 13: In the Azure DevOps window, check organization and project name and click on continue.

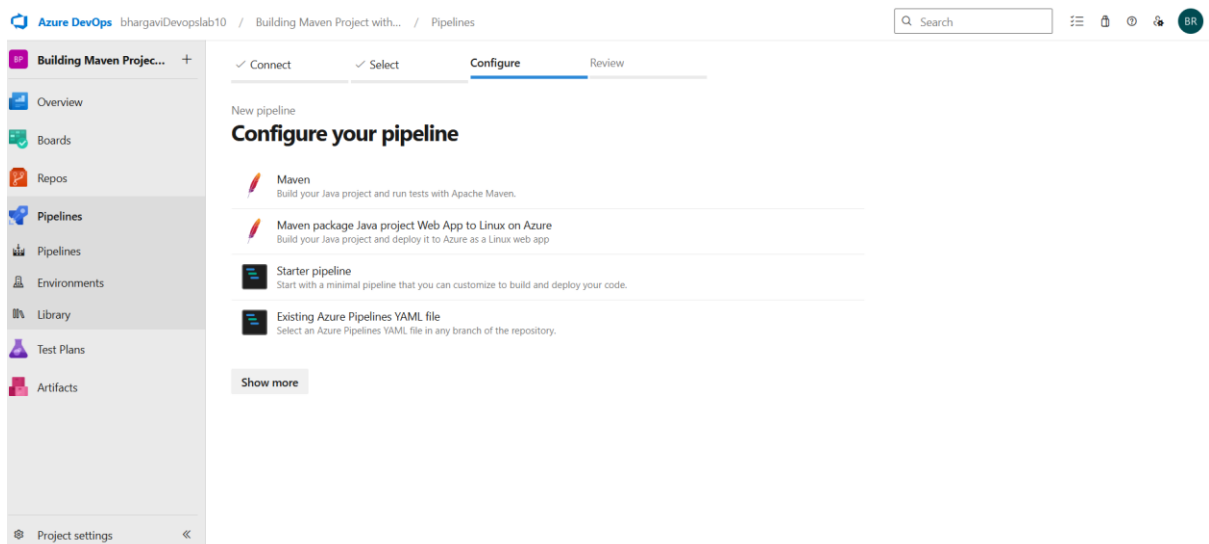


Click on authorize azure pipelines

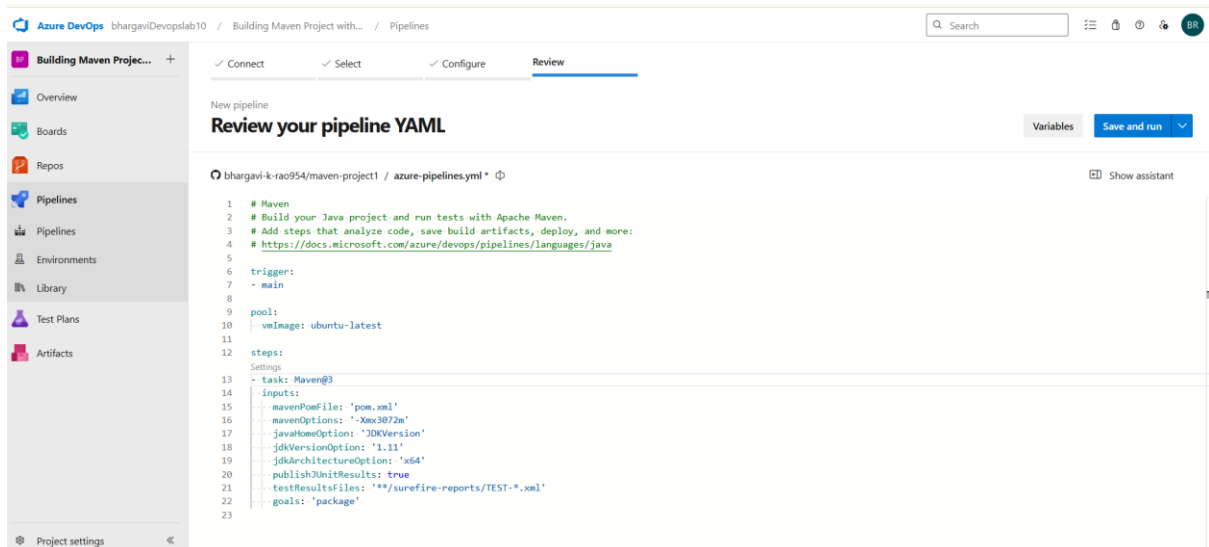


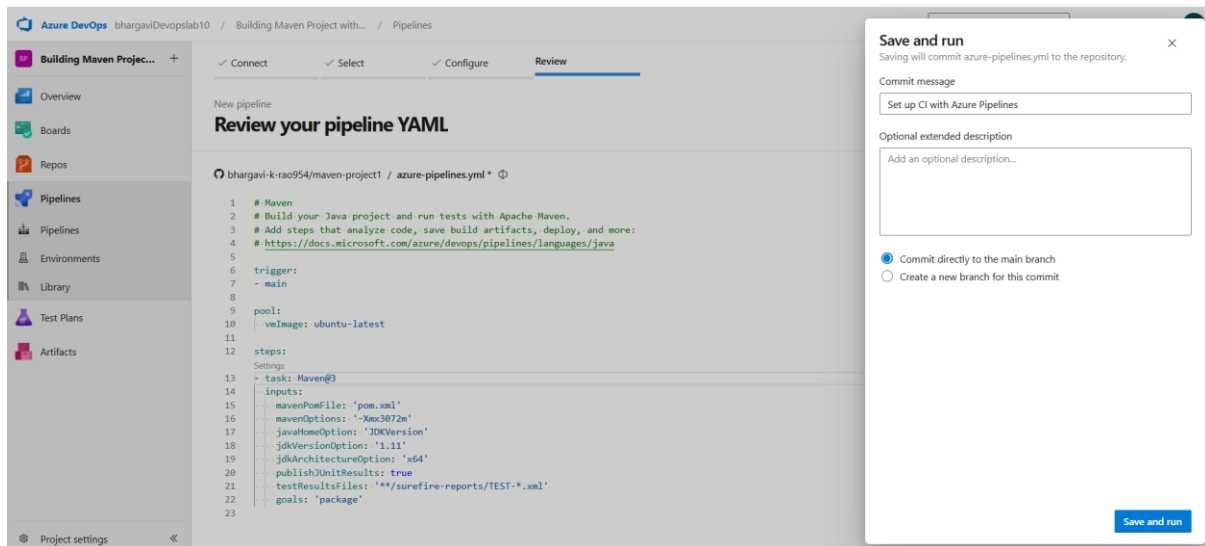


Step 14: You are able to see starter pipeline. select for Maven

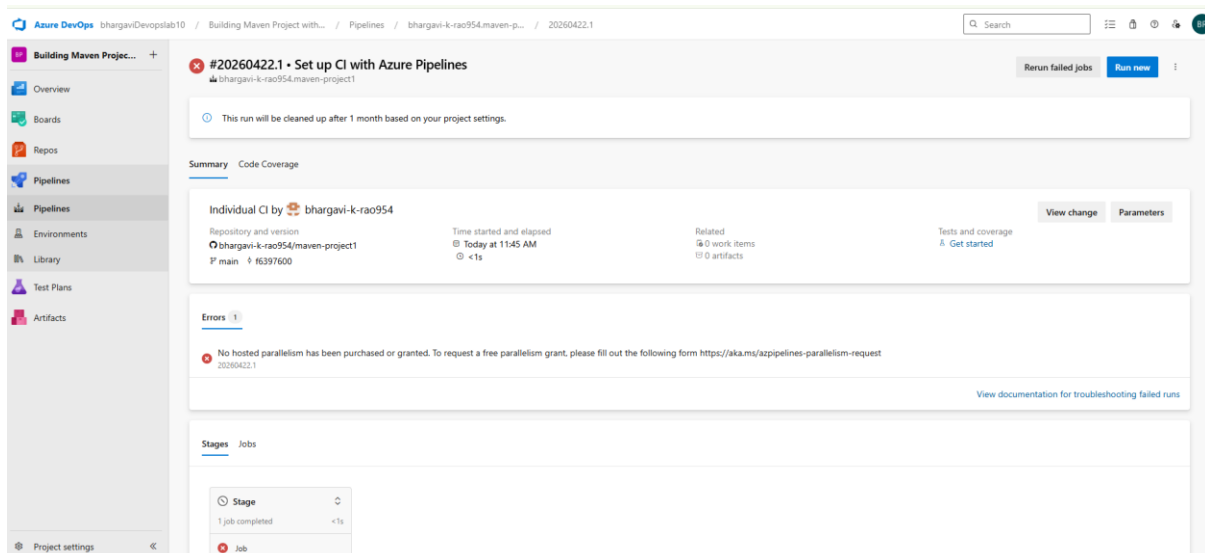


After selecting maven, it asks for save and run just click on it





Step 15: If the run fails with the message like “No hosted parallelism has been purchased” you must request free hosted agent access



Step 16:

Azure DevOps is **refusing to run your pipeline** because:

- Your organization **doesn't have permission** to use Microsoft-hosted agents (virtual machines that run your pipeline).
- By default, **new organizations created after certain limits** must **request access to free parallel jobs**, even for public projects.

Solution:

👉 **Request Free Parallelism**

1. Go to this Microsoft form:
<https://aka.ms/azpipelines-parallelism-request>
2. Fill in the required details:

Azure DevOps Parallelism Request

This form is for users to request increased parallelism in Azure DevOps.

Please consider that it could take 4-5 business days to process the request. We are working on improving this process at the moment. Sorry for the inconvenience.

* Required

1. What is your name? *

Enter your answer

2. What is your email address? *

Enter your answer

3. What is the name of your Azure DevOps Organization? *

(E.g. for <https://myorganization.visualstudio.com> or <https://dev.azure.com/myorganization> link formats - organization name would be "myorganization")

Enter your answer

4. Are you requesting a parallelism increase for Public or Private projects? *

☐ Private

☐ Public

Submit

3. Wait for the approval usually it takes few hours to 2days

Program 11: Deploying Applications to Azure App Services, Managing Secrets and Configuration with Azure Key Vault, Hands-On: Continuous Deployment with Azure Pipelines.

Step 1: Ensure all recommended settings are enabled under Organization Settings to enforce secure and modern pipeline practices.

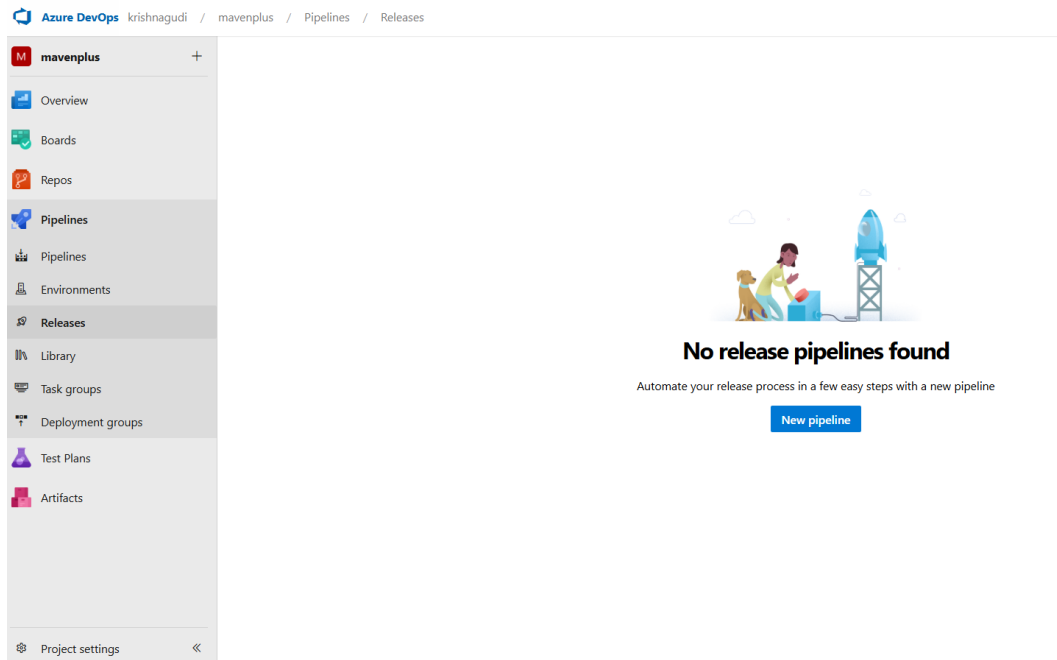
The screenshot shows the Azure DevOps Organization Settings page for 'krishnagudi'. The left sidebar contains a search bar and a list of settings categories: General, Security, Boards, Pipelines, and Agent pools. The main content area displays a list of settings, each with a toggle switch and a description:

- Disable anonymous access to badges** (On): Anonymous users can access the status badge API for all pipelines unless this option is enabled (does not apply to public projects).
- Limit variables that can be set at queue time** (On): You can set any variables at queue time unless this option is enabled. With this option enabled, only those variables that are explicitly marked as "Settable at queue time" can be set. [Learn more](#)
- Limit job authorization scope to current project for non-release pipelines** (On): Non-Release Pipelines can run with collection scoped access tokens unless this option is enabled. With this option enabled, you can reduce the scope of access for all non-release pipelines to the current project.
- Limit job authorization scope to current project for release pipelines** (On): Release pipelines can run with collection scoped access tokens unless this option is enabled. With this option enabled, you can reduce the scope of access for all release pipelines to the current project.
- Protect access to repositories in YAML pipelines** (On): Apply checks and approvals when accessing repositories from YAML pipelines. Also, generate a job access token that is scoped to repositories that are explicitly referenced in the YAML pipeline.
- Disable stage chooser** (Off): With this enabled, users will not be able to select stages to skip from the Queue Pipeline panel.
- Disable creation of classic build pipelines** (On): No classic build pipelines can be created / imported. Existing ones will continue to work.
- Disable creation of classic release pipelines** (Off): No classic release pipelines, task groups, and deployment groups can be created / imported. Existing ones will continue to work.

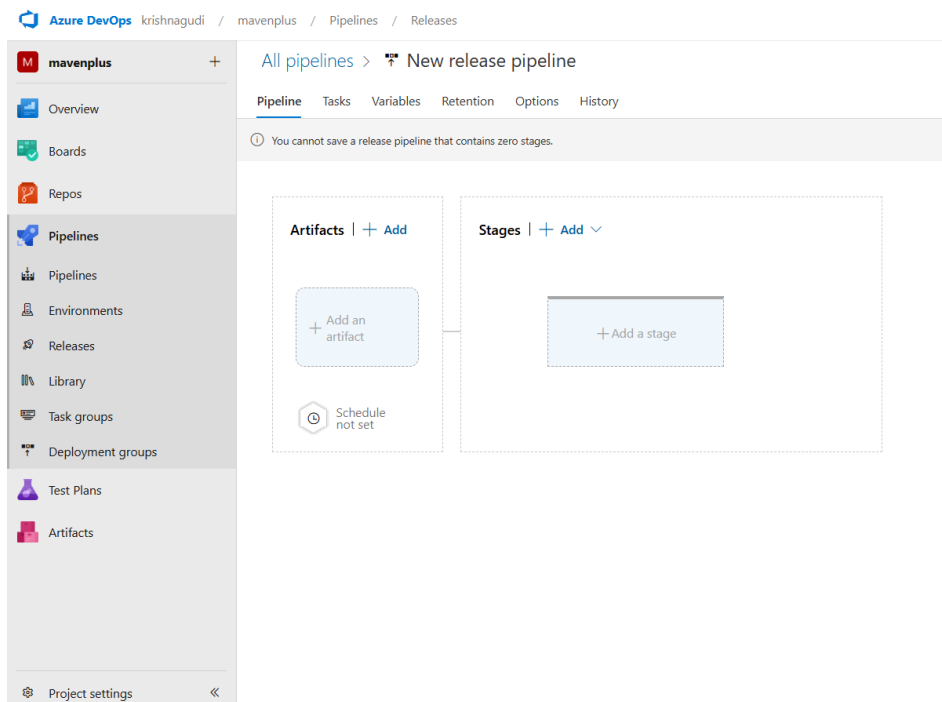
Step 2: After build process, the run failed as no parallelism is granted—students must request free parallelism using the given form.

The screenshot shows the Azure DevOps Pipelines run details page for a failed build. The top navigation bar shows the path: Azure DevOps / krishnagudi / mavenplus / Pipelines / ksgudi.FirstProject / 20250520.1. The left sidebar contains a list of navigation items: Overview, Boards, Repos, Pipelines, Environments, Releases, Library, Task groups, Deployment groups, Test Plans, and Artifacts. The main content area displays the run details for the build #20250520.1, titled 'Update azure-pipelines.yml for Azure Pipelines'. The run was manually run by 'krishna gudi' and failed with the error: 'No hosted parallelism has been purchased or granted. To request a free parallelism grant, please fill out the following form https://aka.ms/azpipelines-parallelism-request'. The page includes a 'Summary' tab, a 'Code Coverage' tab, and a 'View change' button. A 'View documentation for troubleshooting failed runs' link is also present.

Step 3: No release pipeline exists—click on ‘New pipeline’ to start creating one for deployment.



Step 4: Begin your release pipeline by adding an artifact and at least one stage to proceed.



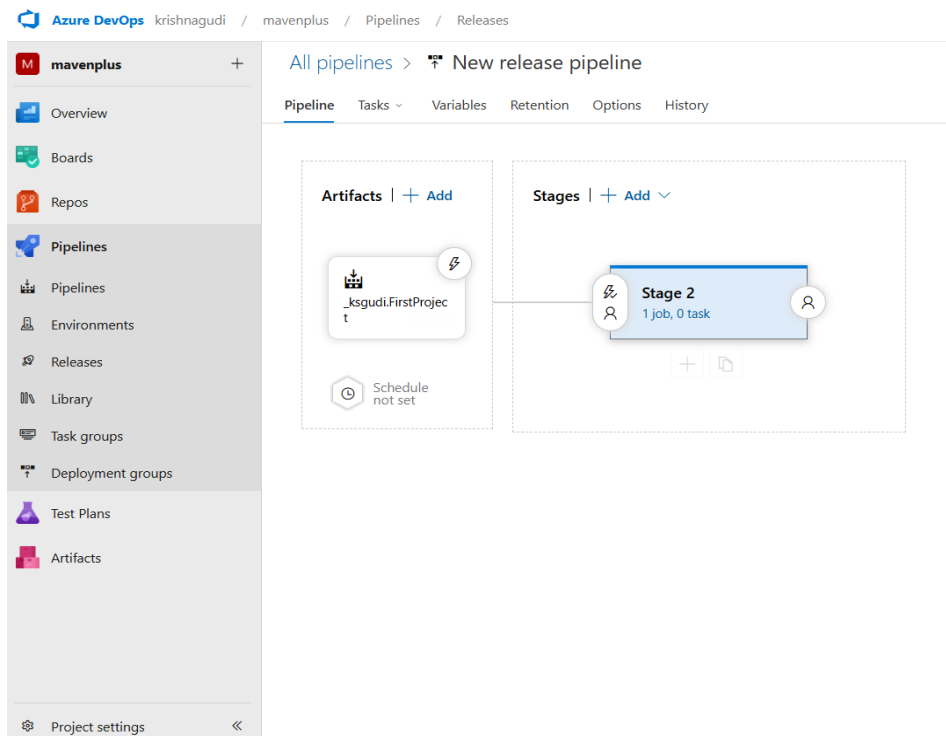
Step 5: Select the build pipeline as the artifact source to link your build output to the release pipeline.

The screenshot shows the Azure DevOps interface for a new release pipeline. The left sidebar contains navigation links for Overview, Boards, Repos, Pipelines, Environments, Releases, Library, Task groups, Deployment groups, Test Plans, and Artifacts. The main area is titled 'New release pipeline' and includes tabs for Pipeline, Tasks, Variables, Retention, Options, and History. A message states: 'You cannot save a release pipeline that contains zero stages.' The 'Artifacts' section shows an 'Add an artifact' button, and the 'Stages' section shows an 'Add a stage' button. On the right, the 'Add an artifact' dialog is open, showing the 'Source type' as 'Build'. The 'Project' is set to 'mavenplus', the 'Source (build pipeline)' is 'ksgudi.FirstProject', and the 'Default version' is 'Latest'. The 'Source alias' is '_ksgudi.FirstProject'. A warning message at the bottom states: 'No version is available for ksgudi.FirstProject or the latest version has no artifacts to publish. Please check the source pipeline.' The 'Add' button is at the bottom right of the dialog.

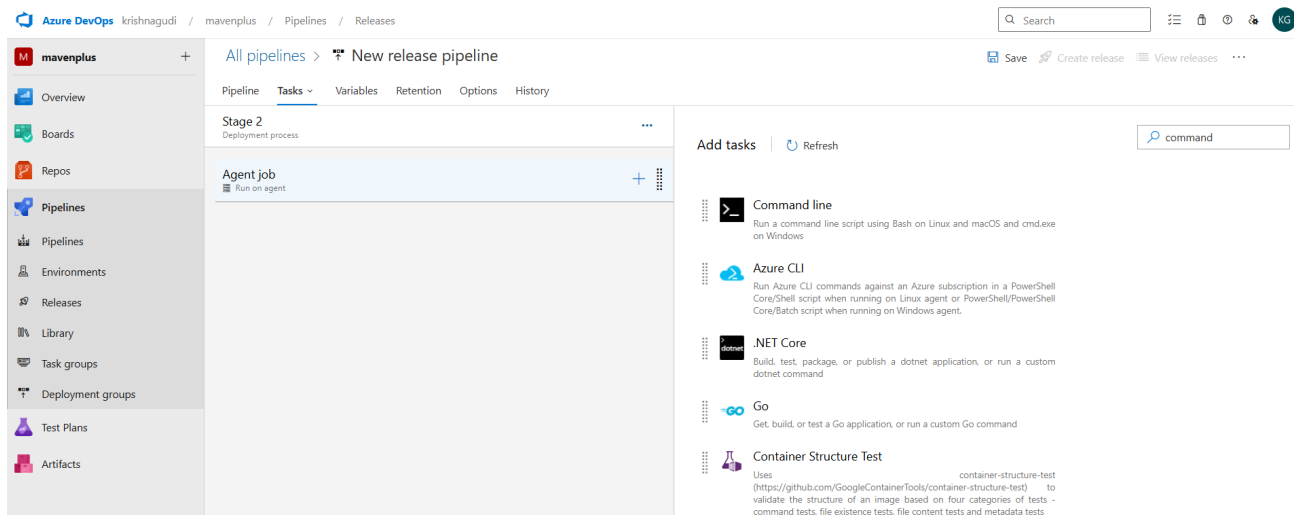
Step 6: After adding the artifact, add a stage and choose a deployment template or start with an empty job.

The screenshot shows the Azure DevOps interface for a new release pipeline. The left sidebar is the same as in Step 5. The main area shows the 'New release pipeline' with the 'Artifacts' section now containing '_ksgudi.FirstProject' and the 'Stages' section containing 'Stage 2' with a 'Select a template' button. On the right, the 'Select a template' dialog is open, showing a search bar and a list of featured templates. The 'Featured' section includes: 'Azure App Service deployment', 'Deploy a Java app to Azure App Service', 'Deploy a Node.js app to Azure App Service', 'Deploy a PHP app to Azure App Service and Azure Database for MySQL', 'Deploy a Python app to Azure App Service and Azure database for MySQL', 'Deploy to a Kubernetes cluster', and 'IIS website and SQL database deployment'. The 'Others' section is also visible at the bottom.

Step 7: Stage is created—click to configure jobs and tasks for deployment.



Step 8: Under ‘Tasks’, add a new deployment task such as a command line script to simulate or execute deployment steps.



Step 9: In the command line task, enter deployment commands to simulate deployment or perform actual actions.

The screenshot shows the Azure DevOps web interface. The left sidebar contains navigation links: Overview, Boards, Repos, Pipelines, Environments, Releases, Library, Task groups, Deployment groups, Test Plans, and Artifacts. The main area is titled 'New release pipeline' and shows 'Stage 2' with a 'Deployment process' job. A task named 'Command Line Script' is selected. The right-hand configuration pane for this task shows 'Task version' set to '2.*'. The 'Display name' is 'Command Line Script'. The 'Script' field contains the text: 'echo "Simulated Deployment Step"' and 'echo "Artifact would be deployed here"'. Below the script field are expandable sections for 'Advanced', 'Control Options', 'Environment Variables', and 'Output Variables'.

Simulate Secrets (Key Vault):

Step 10: Click on '+ Variable group' to create a new group of shared pipeline variables.

The screenshot shows the 'Library' page in Azure DevOps. The left sidebar is the same as in the previous image. The main area has tabs for 'Variable groups', 'Secure files', '+ Variable group', 'Security', and 'Help'. The 'Variable groups' tab is active. In the center of the page is a large graphic with a box containing '(x)'. Below this graphic, the text reads 'New variable group' and 'Create groups of variables that you can share across multiple pipelines.' There is a blue button labeled '+ Variable group' and a link that says 'Learn more about variable groups.'.

Step 11: Enter a name and define secure variables (e.g., DB_PASSWORD) for use across multiple pipelines.

Azure DevOps krishnagudi / mavenplus / Pipelines / Library

Library > New variable group 20-May*

Variable group | Save | Clone | Security | Pipeline permissions | Approvals and checks | Help

Properties

Variable group name
New variable group 20-May

Description

☐ Link secrets from an Azure key vault as variables ⓘ

Variables

Name ↑	Value	
name (required)		

+ Add

Step 12: Fill in variable name and value, and click Save to finalize the variable group setup.

Azure DevOps krishnagudi / mavenplus / Pipelines / Library

Library > New variable group 20-May*

Variable group | Save | Clone | Security | Pipeline permissions | Approvals and checks | Help

Properties

Variable group name
New variable group 20-May

Description

☐ Link secrets from an Azure key vault as variables ⓘ

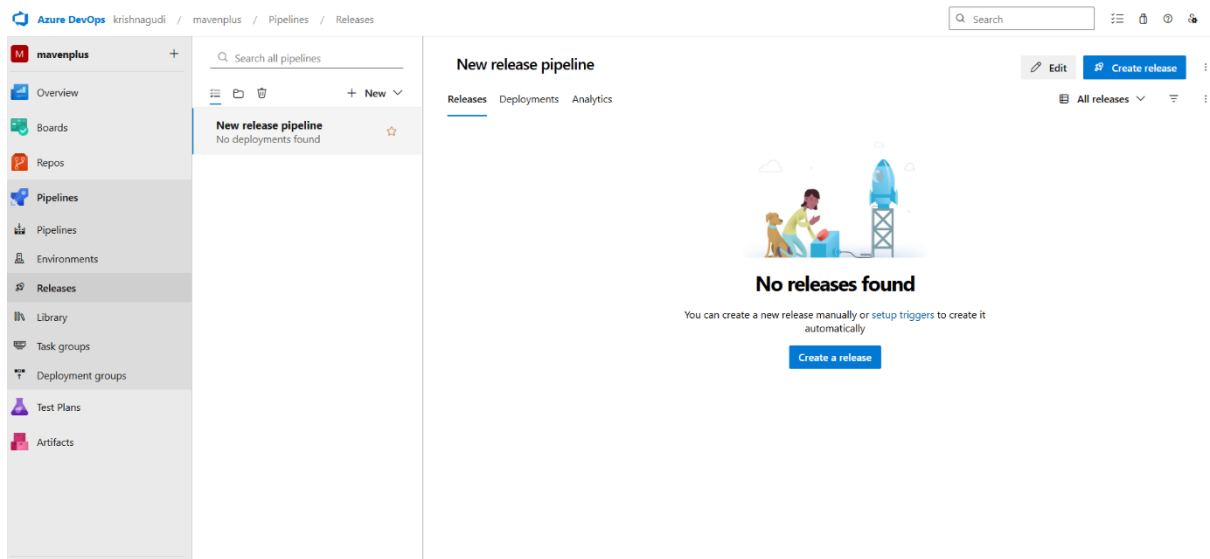
Variables

Name ↑	Value	
DB_PASSWORD	xyz123	

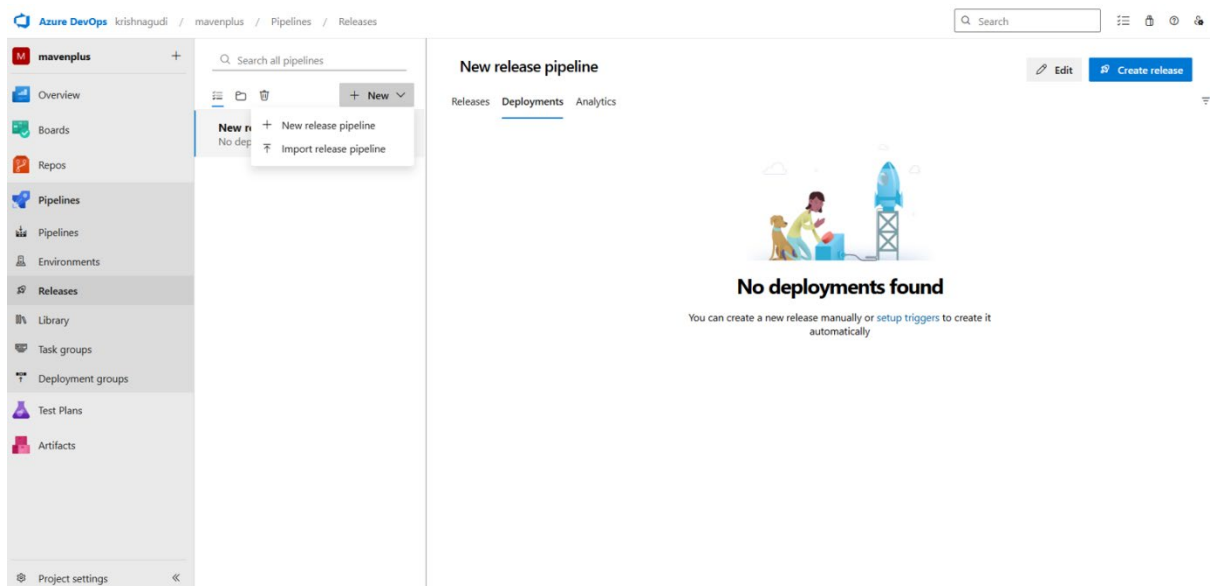
+ Add

Link this variable group in your Release Pipeline:

Step 13: Select the created variable group and click ‘Link’ to use it in the release pipeline.



Step 14: Click on ‘Create a release’ to trigger the release pipeline manually.



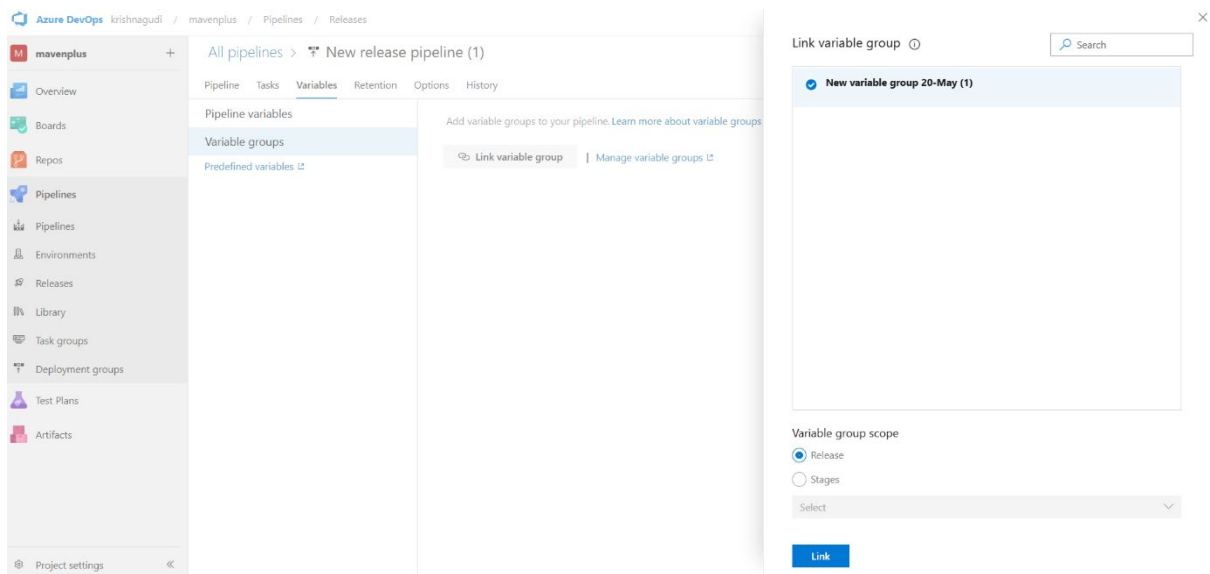
Step 15: Click the ‘+ New’ button to create or import a release pipeline when none exists.

The screenshot shows the Azure DevOps interface for a project named 'mavenplus'. The left sidebar contains a navigation menu with options like Overview, Boards, Repos, Pipelines, Environments, Releases, Library, Task groups, Deployment groups, Test Plans, and Artifacts. The main area is titled 'New release pipeline (1)' and has tabs for Pipeline, Tasks, Variables, Retention, Options, and History. The 'Variables' tab is selected, displaying 'Pipeline variables' and 'Variable groups'. A search bar with the text 'Filter by keywords' and a 'Scope' dropdown is at the top. Below it is a table with columns 'Name' and 'Value'. A '+ Add' button is located at the bottom of the table.

Step 16: Click ‘+ Add’ under Pipeline variables to define new custom key-value variables.

The screenshot shows the Azure DevOps interface for a project named 'mavenplus'. The left sidebar contains a navigation menu with options like Overview, Boards, Repos, Pipelines, Environments, Releases, Library, Task groups, Deployment groups, Test Plans, and Artifacts. The main area is titled 'New release pipeline (1)' and has tabs for Pipeline, Tasks, Variables, Retention, Options, and History. The 'Variables' tab is selected, displaying 'Pipeline variables' and 'Variable groups'. A search bar with the text 'Filter by keywords' and a 'Scope' dropdown is at the top. Below it is a table with columns 'Name' and 'Value'. A '+ Add' button is located at the bottom of the table.

Step 17: Use ‘Link variable group’ to connect predefined variables to your release pipeline.



“To overcome subscription-related limitations, a simulated deployment approach was adopted using command line scripts. This enabled demonstration of the complete release pipeline flow without requiring actual agent execution. It served as an effective workaround in a restricted environment.”

Program 12: Build and Deploy a Complete DevOps Pipeline, Discussion on Best Practices and Q&A.

- Build Pipeline (CI) compiles the code, runs tests, and prepares an artifact (e.g., .jar, .zip) when code is pushed.
- Release Pipeline (CD) takes that artifact and deploys it to a target environment (e.g., Azure App Service).
- Together, it ensures smooth, continuous delivery from code commit to deployment with minimal manual steps.

Build Pipeline (CI) Setup:

Step 1: Go to Azure DevOps → Pipelines → Create Pipeline

Step 2: Select GitHub as the code repository and choose your sample Maven project.

Step 3: Choose **YAML** and create the following build steps:

- Checkout code
- Maven build task (mvn clean install)
- Publish artifact (e.g., JAR or ZIP)

Step 4: Commit and run the pipeline.

Step 5: If **Hosted Parallelism is not approved**, you may simulate steps using:

```
script: echo "Simulating Maven Build..."
script: echo "Simulating Artifact Publish..."
```

Release Pipeline (CD) Setup:

Step 6: Go to Pipelines → **Releases** → **New Pipeline**

Step 7: Add an Artifact → Select the build pipeline output.

Step 8: Add a Stage → Choose “**Empty Job**”

Step 9: In the Tasks tab, add a Command Line Script task:

```
echo "Simulating deployment to Azure App Service..."  
echo "Deploying artifact: HelloWorld-1.0.jar"
```

Step 10: Save and create a release → Manually trigger it to run.

Step 11: Use screenshots to validate artifact linkage and simulation.

Simulating Secrets with Variable Groups:

Step 12: Go to Pipelines → Library → + Variable Group

Step 13: Name it (e.g., MySecretsGroup), add secure variables (e.g., DB_PASSWORD)

Step 14: Save the group and link it in your release pipeline (under Variables tab).

-----*****-----

Discussion on Best Practices (Examples)

- Use version control for all YAML/pipeline files.
- Simulate builds when agents are unavailable.
- Store secrets securely in variable groups or Key Vault.
- Break pipelines into clear stages (build, test, deploy).
- Always review logs and test locally before release.